



并行计算

Parallel Computing

张超

中国人民大学信息学院



课程介绍

- 三位老师联合授课
 - 张超 老师: **1-4周 【并行CPU编程】**
 - 谢旻辉 老师: **6-10周 【并行CUDA编程】**
 - 鄂金龙 老师: **11-14 周 【分布式计算】**
 - 张超 老师: **15-16周 【前沿研讨与作业回顾】**
- 四位经验助教
 - 冯琳: **2024级直博**
 - 周智淦: **2024级学硕**
 - 崔钰锟: **2024级学硕**
 - 周强俊: **2024级学硕**



课程介绍

- 成绩构成
 - 70%平时成绩、30%期末考核
 - 平时成绩：50%课程作业、50%课堂表现
 - 期末考核：以论文写作形式完成
 - （2页英文实验报告，介绍一项并行计算技术与实验）
 - 不要抄袭！不要抄袭！不要抄袭！
 - 作弊/抄袭即开除！

论文样例

Parallel Processing in LU Decomposition

Abstract—LU decomposition is a crucial technique in numerical linear algebra, widely utilized in numerical computation technologies such as solving linear equations, finding matrix inverses, or calculating determinants. As the matrix dimension increases, the complexity of LU decomposition grows polynomially. To address the escalating demand for efficient computations, this study investigates the acceleration of LU decomposition on both CPU and GPU platforms. The research employs CPU parallelization with OpenMP and GPU acceleration with CUDA, followed by performance evaluations, offering insights into enhancing parallel accelerated LU decomposition algorithms.

I. INTRODUCTION

LU decomposition is a matrix factorization method in linear algebra that decomposes a matrix into the product of a lower triangular matrix L and an upper triangular matrix U, as shown in Fig. 1. This technique is primarily applied in numerical analysis for solving linear equations, finding matrix inverses or calculating determinants.

$$A = \begin{bmatrix} \blacksquare & 0 & 0 & 0 & 0 \\ \blacksquare & \blacksquare & 0 & 0 & 0 \\ \blacksquare & \blacksquare & \blacksquare & 0 & 0 \\ \blacksquare & \blacksquare & \blacksquare & \blacksquare & 0 \\ \blacksquare & \blacksquare & \blacksquare & \blacksquare & \blacksquare \end{bmatrix} = \begin{bmatrix} \blacksquare & \blacksquare & \blacksquare & \blacksquare & \blacksquare \\ 0 & \blacksquare & \blacksquare & \blacksquare & \blacksquare \\ 0 & 0 & \blacksquare & \blacksquare & \blacksquare \\ 0 & 0 & 0 & \blacksquare & \blacksquare \\ 0 & 0 & 0 & 0 & \blacksquare \end{bmatrix}$$

Fig. 1. LU Decomposition 1

Furthermore, we aim for the diagonal elements of the lower triangular matrix to be equal to 1, as shown in Fig. 2.

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ \blacksquare & 1 & 0 & 0 & 0 \\ \blacksquare & \blacksquare & 1 & 0 & 0 \\ \blacksquare & \blacksquare & \blacksquare & 1 & 0 \\ \blacksquare & \blacksquare & \blacksquare & \blacksquare & 1 \end{bmatrix} = \begin{bmatrix} \blacksquare & \blacksquare & \blacksquare & \blacksquare & \blacksquare \\ 0 & \blacksquare & \blacksquare & \blacksquare & \blacksquare \\ 0 & 0 & \blacksquare & \blacksquare & \blacksquare \\ 0 & 0 & 0 & \blacksquare & \blacksquare \\ 0 & 0 & 0 & 0 & \blacksquare \end{bmatrix}$$

Fig. 2. LU Decomposition 2

II. BACKGROUND OF LU METHODS

A. The Algorithm of LU Decomposition

LU decomposition employs the method of elimination, progressively breaking down the matrix A as illustrated in Fig. 3.

$$\begin{bmatrix} \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix} \xrightarrow{L_2} \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ \times & \times & \times & \times \\ \times & \times & \times & \times \end{bmatrix} \xrightarrow{L_3} \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ \times & \times & \times & \times \end{bmatrix} \xrightarrow{L_4} \begin{bmatrix} \times & \times & \times & \times \\ 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & 0 & \times \end{bmatrix}$$

$A \quad L_2 A \quad L_3 L_2 A \quad L_4 L_3 L_2 A = U$

Fig. 3. breaking down the matrix A

By using the elimination method, we can transform matrix A into an upper triangular matrix U with the help of the $L_0 L_1 L_2$ matrix, i.e., $U = L_2 L_1 L_0 A$. Therefore, $L = L_0^{-1} L_1^{-1} L_2^{-1}$. The computation of the inverse matrix involves simply reversing the signs of the corresponding matrix elements.

Here is the flowchart illustrating the algorithmic process of LU decomposition.

Algorithm 1 LU Decomposition

```

for i = 0 to n do
    Lii = 1
    for j = i to n do
        Uij = Aij
        for k = 0 to i do
            Uij = Uij - Lik * Ukj
        end for
    end for
    for j = i + 1 to n do
        Lij = Aji
        for k = 0 to i do
            Lji = Lij - Ljk * Uki
        end for
        Lji = Lji / Uii
    end for
end for

```

B. The complexity of LU Decomposition

Due to the triple-nested loops involved, the time complexity of LU decomposition is $O(n^3)$, where the complexity of addition operations is $\frac{1}{3}n^3$, the complexity of multiplication operations is $\frac{2}{3}n^3$, so the overall complexity is $\frac{2}{3}n^3$.

III. EXPERIMENT

In the section I will describe a few of my attempts to speedup LU decomposition on CPU and GPU, and show the performance of the program.

A. CPU Version1: Simple

In this version, I didn't do any optimization, just implement it according to the above algorithm. And due to the high computational complexity, it performs badly, even in -O3 optimization, it can only achieve an average of 0.406 GFLOPS/s

B. CPU Version1: OpenMP

By parallelizing the core components of LU decomposition using OpenMP, it achieve an average of 1.175 GFLOPS/s, approximately 3x faster than Version1.

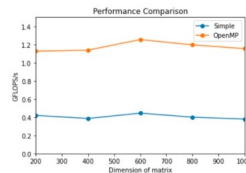


Fig. 4. CPU GFLOPS/s from 2080h

C. GPU Version1: Simple

In this version, I didn't implement any optimization; I simply converted the code to CUDA to run on the GPU. Due to the complexity of the code, parallelization wasn't straightforward, and the optimization effect wasn't very pronounced. In cases where the matrix dimension is less than 1000, the achieved performance reached 2.296 GFLOPS/s, which is twice that of the CPU parallel version.

D. GPU Version2: Code Refactoring

In this version, I focused on the code structure and performed loop merging and loop unrolling while ensuring correctness. I separated the assignment of the A matrix to the A and U matrices into one function, and the update of the A and U matrices into another function. I optimized the loops inside these functions through loop merging and loop unrolling. In this version, the achieved performance reached 11.846 GFLOPS/s when the matrix dimension is less than 1000. The parallel acceleration effect has significantly improved.

E. GPU Version3: Using Shared Memory

In this version, I employed GPU shared memory to accelerate the code. It is evident that the performance improvement is substantial, and the code exhibits excellent performance even with large-dimensional matrices.

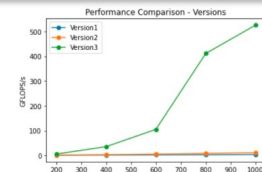


Fig. 5. GPU GFLOPS/s from 2080ti

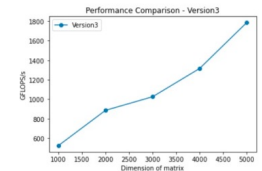


Fig. 6. GPU GFLOPS/s from 2080ti

F. Analysis

It can be observed that the code has undergone parallelization restructuring, reducing the non-parallelizability of the code. Additionally, with the optimization using shared memory, the parallel performance has achieved an acceleration of over 100 times, and the performance increases with the growth of matrix dimensions.

G. Accuracy

After computing the matrix L and U, I compute $\delta = \max(|A - L \cdot U|)$ to check the accuracy of the algorithm. And when using single precision, the maximum error is about 10^{-5} and when using double type, the error is about 10^{-15} . So this method is able to provide a reliable result.

IV. CONCLUSION

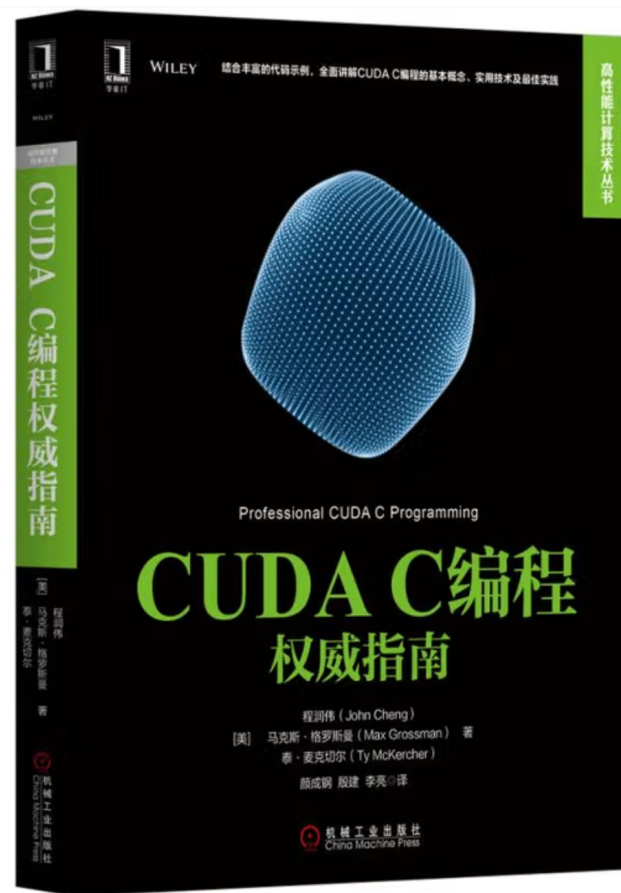
In this paper, I introduced the algorithm of LU decomposition and parallelized its implementation on both CPU and GPU. While ensuring correctness, I achieved acceleration from 0.4 GFLOPS/s to 100+ GFLOPS/s, offering insights into enhancing parallel accelerated LU decomposition algorithms.



课程目的

- 掌握并行算法的基本概念和设计分析方法
- 熟知基本的并行程序设计方法
- 理论知识与动手实践并重，编程解决问题
- 了解前沿进展，提出自己的洞见与idea

书籍推荐





教学大纲

教学周	章节名称	教学内容与教学要求	课外学习内容与要求
第1周	并行体系结构基础	现代处理器	放在OBE网站上同学们可以自行下载学习
第2周	CPU并行编程	(1) 并行计算机和编程模型的分类; (2) 超级计算机和最新进展; (3) 并行政程序分析。	对上周的作业进行了总结
第3周	多线程OpenMP编程	(1) 如何在Linux中定义线程; (2) 使用OpenMP进行线程并行; (3) 如何检测数据竞争。	对上周的作业进行了总结
第4周	OpenMP编程优化	在介绍OpenMP具体语法后, 需要进一步了解如何对OpenMP程序进行优化获得更大的性能提升。	(1) 串行程序的性能优化; (2) 并行程序的性能优化; (3) OpenMP新特性
第5周	国庆假期		
第6周	GPU介绍	本课程主要对GPU进行介绍, 让同学们对异构体系结构有一个整体的了解。	(1) GPU的兴起; (2) GPU加速与高性能计算; (3) NVIDIA CUDA简介。
第7周	CUDA编程入门	本课程主要对GPU CUDA编程进行讲解, 同学们学完后可以进行基础的GPU编程。	(1) GPU的线程; (2) GPU的内存使用; (3) GPU上的同步。
第8周	CUDA程序优化	本课程主要对GPU CUDA的相关性能优化因素进行讲解。	(1) GPU 内存聚合性; (2) GPU的共享内存使用; (3) GPU占用率调优
第9周	CUDA实例分析	本课程通过对特有的异构问题讲解使得同学们对GPU编程有更深入的认识。	(1) 性能考虑因素; (2) 稀疏矩阵向量乘 (SpMV) ; (3) 稀疏下三角矩阵求解 (SpTRSV)
第10周	CUDA程序分析	本课程通过对GPU程序分析达到对GPU有更好了解的目的。	(1) 性能计数器统计; (2) GPU常用分析工具; (3) GPU程序优化流程。
第11周	分布式计算概述	分布式基本概念, 了解; 分布式的发展历史, 了解	阅读教材相关章节, 熟练掌握相关知识;
第12周	分布式核心技术		
第13周	云计算技术	熟练掌握云计算核心技术	阅读教材相关章节, 熟练掌握相关知识
第14周	前沿学术探讨		
第15周	前沿学术探讨	论文研读	阅读教材相关章节, 熟练掌握相关知识
第16周	作业回顾	作业讲解	问答环节与汇报

教学进度安排



1-4周课程规划

- 第一周：并行计算概述
- 第二周：OpenMP编程（Multiprocessing）
- 第三周：MPI编程 (Message Passing Interface)
- 第四周：HTAP数据库

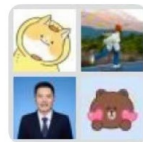


5-6周课程规划

- 第15周： 前沿讨论
 - GPU数据库
- 第16周： 作业回顾
 - 作业总体评价
 - 作业点评



课程二维码



群聊: 并行计算 2025



该二维码 7 天内 (9月15日前) 有效, 重新进入将更新



目录

- 并行计算介绍
 - 什么是并行计算
 - 我们为什么需要并行计算
- 并行计算机和编程模型分类
- 并行计算基本概念
- 并行程序分析



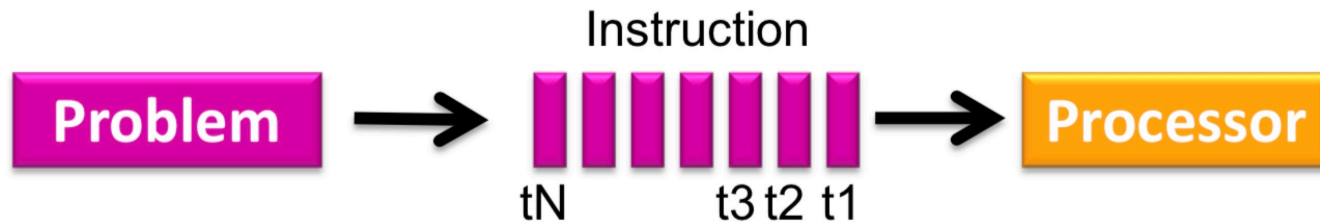
思考



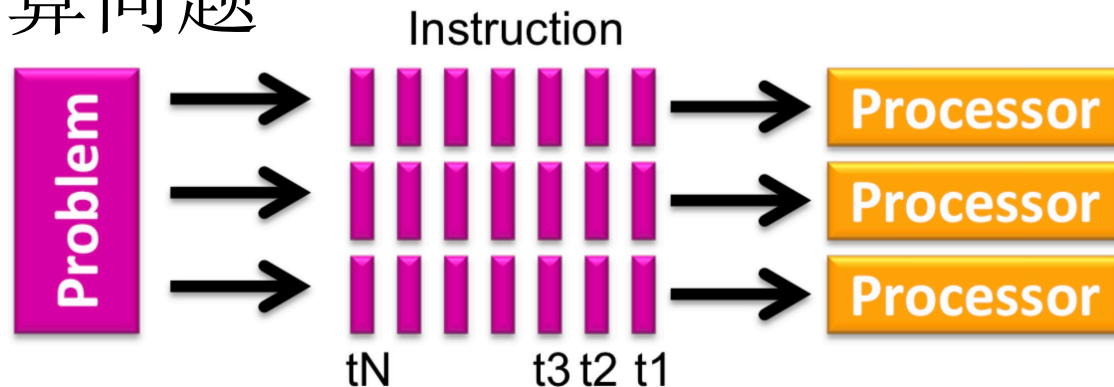
- 什么是并行计算？
- 为什么需要并行计算？

什么是并行计算？

- 通过使用多个处理器（核心）一起工作解决某个问题（single problem）
- 传统编程串行执行计算程序



- 并行计算中，使用多种计算资源一起解决某个计算问题



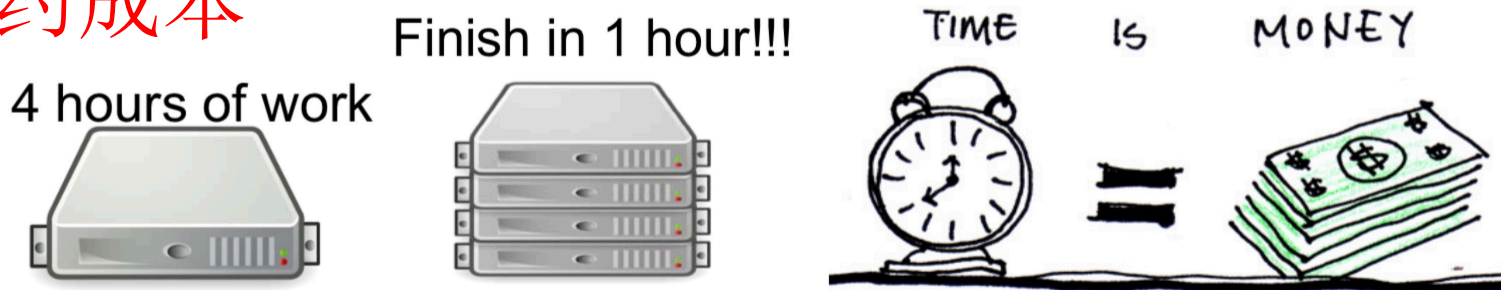


并行计算和分布式计算的区别

- 两个概念非常相近，但来自两个不同方面
- Parallel computing
 - 不同的计算活动在**同一时间**发生
 - **单个应用**使用多核/多处理器/多进程 来处理更大规模负载或同样规模负载算得更快
 - 通常用于**科学计算**
- Distributed computing
 - **跨系统或不同位置的服务器**
 - 更专注于**并发控制**和**资源共享**
 - 通常来自于**商业领域**

为什么需要并行计算

- 节省时间
- 使用更多的资源来减少运行时间，进而节约成本

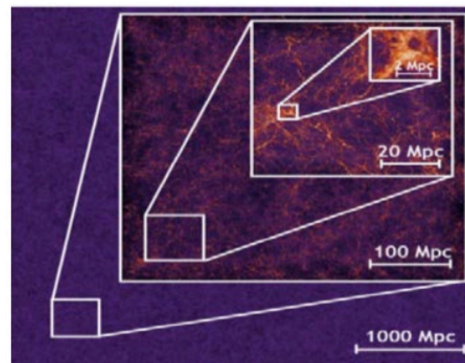


- 更短的运行时间可以执行更多次，调程序更多次

	DUAL XEON CPU server	DGX-1 GPU server (8 GPUs)
FLOPS	3TF	170TF
Node Mem BW	76GB/s	768GB/s
Alexnet Train Time	150 Hr	2Hr
Train in 2Hr	>250Nodes	1Node

为什么需要并行计算

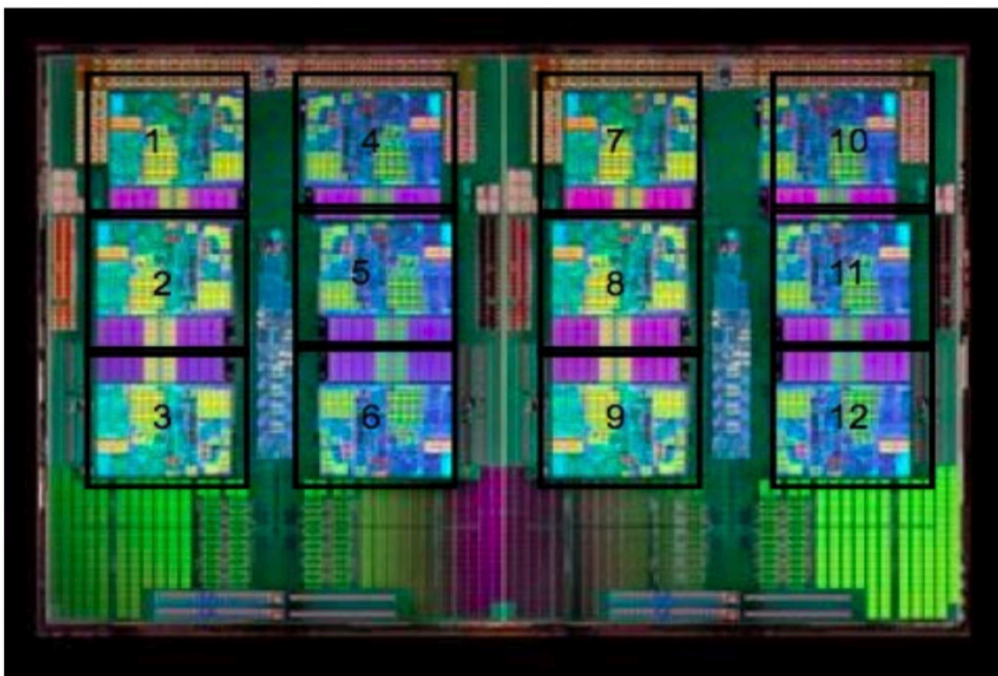
- 解决更大规模的问题
 - 在**单机**上解决**不可能/不实际**
 - 科学计算
 - Trillion级粒度
 - 成千上万参数调节
 - TB级需要处理的数据
 - Million级（PFlops）个核数的集群执行数个小时



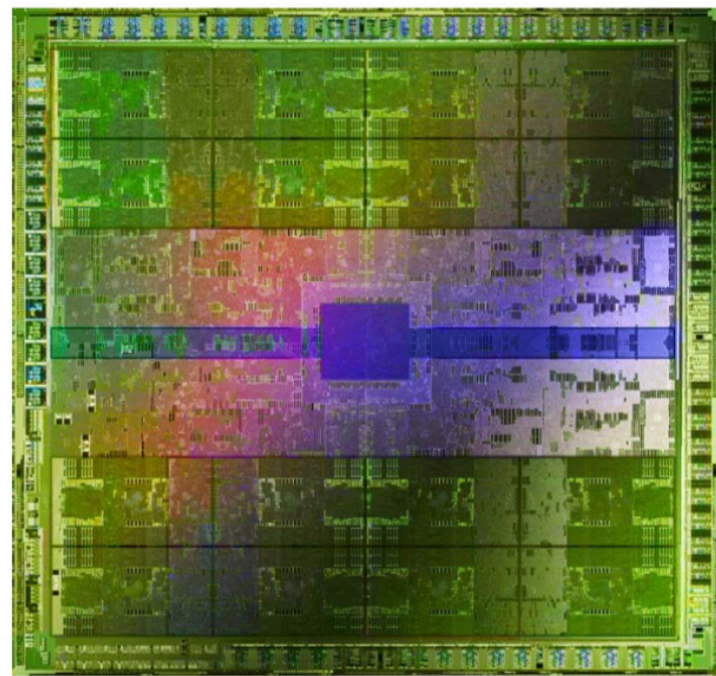
1 trillion particles, 4.225 Gpc box-size simulation, and 6 kpc force resolution.

为什么需要并行计算

- 更好利用并行硬件的潜在性能



12 Cores IBM Blade Multi-core CPU

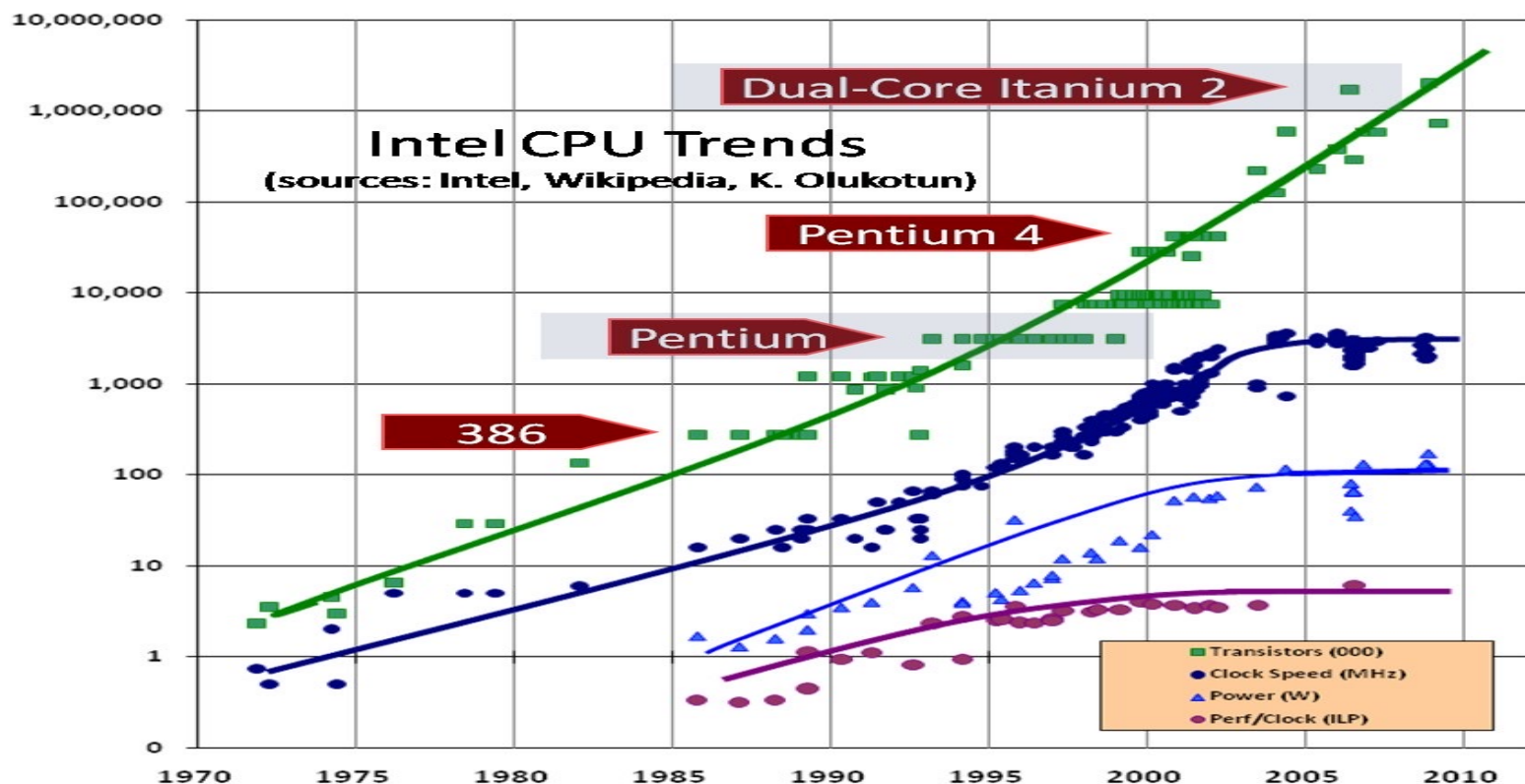


512 Cores NVIDIA Fermi GPU



CPU scaling的终结

- 增加晶体管的密度不等于性能的提升
 - 功耗和时钟频率加速停滞了





并行计算的趋势

Single-Core Era

Enabled by:
Moore's Law
Voltage Scaling

Constraint by:
Power
Complexity

Assembly → C/C++ → Java ...



Multi-Core Era

Enabled by:
Moore's Law
SMP

Constraint by:
Power
Parallel SW
Scalability

Pthread → OpenMP ...



Heterogeneous Systems Era

Enabled by:
Abundant data
parallelism
Power efficient GPUs

Constraint by:
Programming
models
Comm. overhead

Shader → CUDA → OpenCL ...



Distributed System Era

Enabled by:
Networking

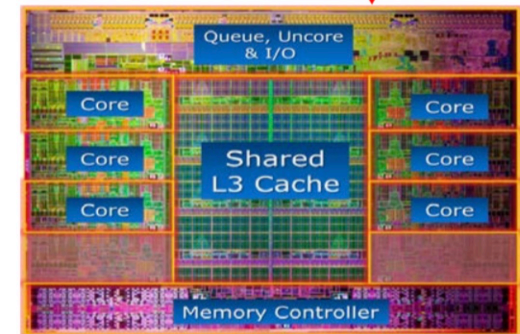
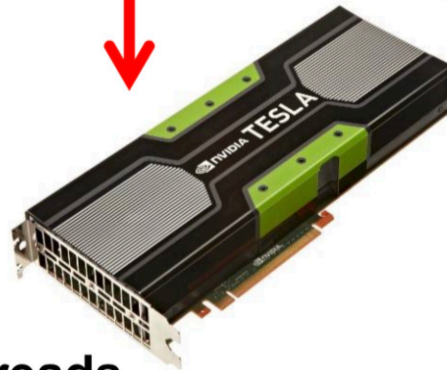
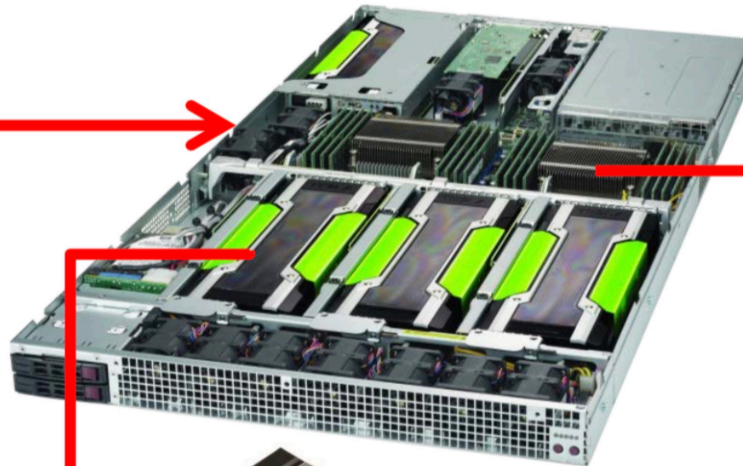
Constraint by:
Synchronization
Comm. overhead

MPI → MapReduce ...

今天的典型并行计算机

Racks: 16~42U

Node/Server: 1~4U



100x cores/1000xthreads

4~12 cores



Supercomputers

- 定义：和通用计算机相比，拥有更强大的计算能力
- 性能由每秒能够执行的浮点指令数（floating- point operations per second, **FLOPS**）衡量，而不是每秒执行的指令数（million instructions per second, MIPS）
- 1993年提出了TOP500
 - **HPL Benchmark**
 - ISC和SC会议两年一次



HPL Benchmark

- Linpack library的并行实现
 - 衡量浮点运算效率
- 计算：
 - 解决线性矩阵求解问题

$$Ax = b; \quad A \in \mathbf{R}^{n \times n}; \quad x, b \in \mathbf{R}^n$$

- LU分解
- 将矩阵分为许多子矩阵
- 所有的参数由用户决定

What makes it a supercomputer

- 什么构成了一台超级计算机？
 - 最新的硬件技术
 - 定制的系统配置
 - 优化的软件和功能库
 - 投入大量的资金，消耗大量能源
- 超级计算机代表着国家层面的技术和财富的竞争





TOP500 List (2016 June)

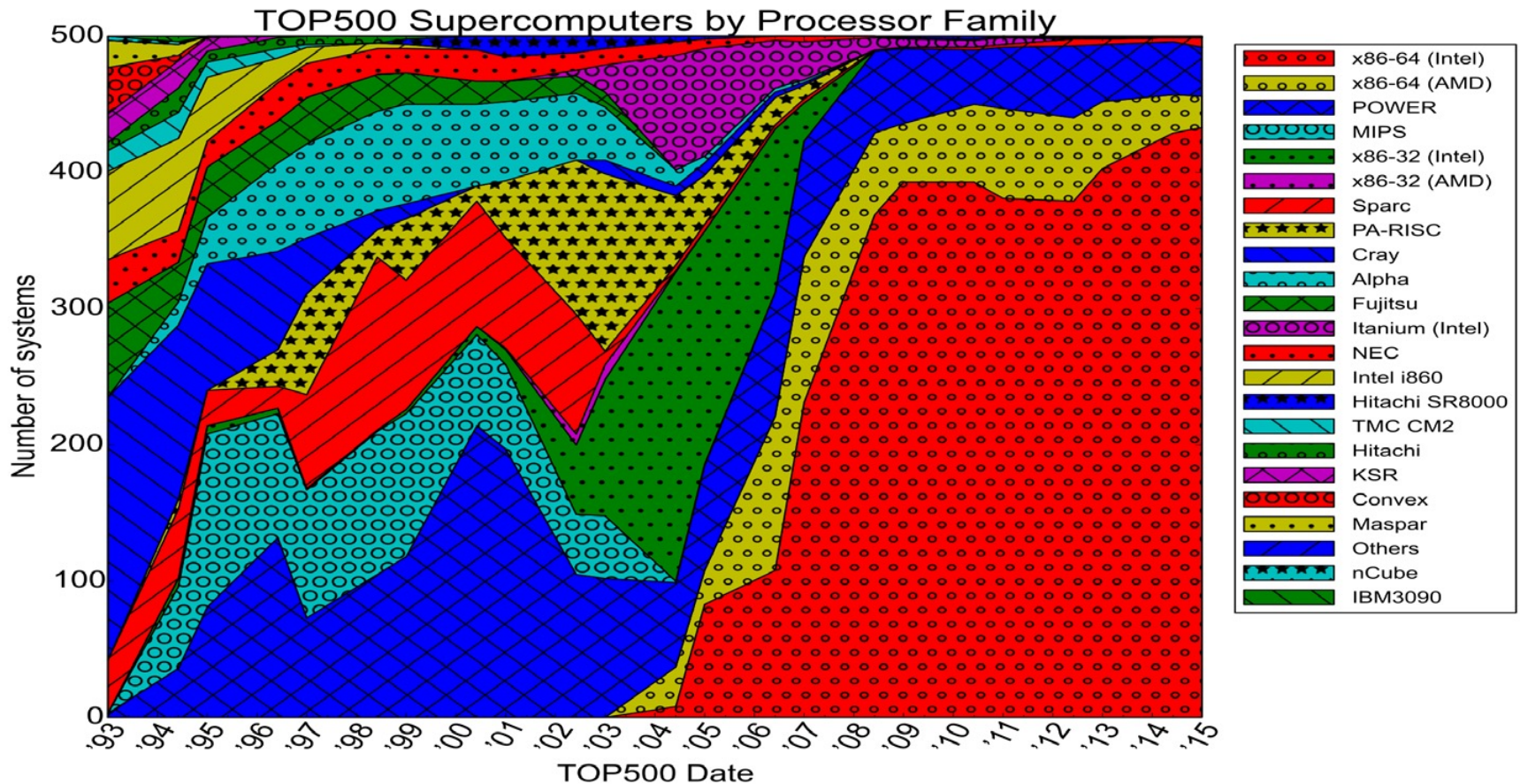
	Country	System	Vendor	Power (kW)	#cores	Accelerator	Rmax	Rpeak (PFLOPS)
1	China	TaihuLight	NRCPC	15,371	10M		93.0	125.4
2	China	Tianhe-2	NUDT	17,808	3M	Xeon Phi	33.9	54.9
3	US	Titan	Cray	8,209	560K	Tesla K20X	17.6	27.1
4	US	Sequoia	IBM	7,890	1.5M		17.2	20.1
5	Japan	K	Fujitsu	12,660	705K		10.5	11.3
6	US	Mira	IBM	3,954	786K		8.6	10.0
7	US	Trinity	Cray		301K		8.1	11.1
8	Swiss	Piz Daint	Cray	2,325	116K	Tesla K20X	6.2	7.8

加速卡提供了强大的计算能力

Titan的Rmax数值如果没有GPU只有2K

TOP500 Trend: CPU

- Intel CPU 占80%以上

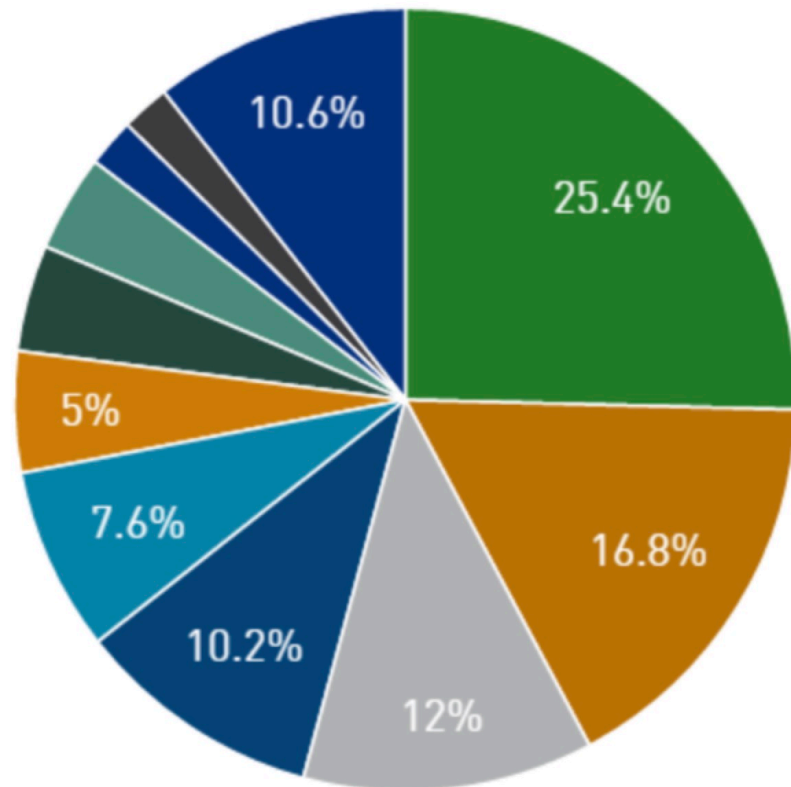




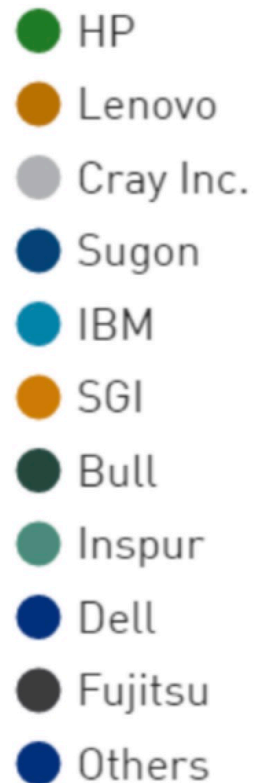
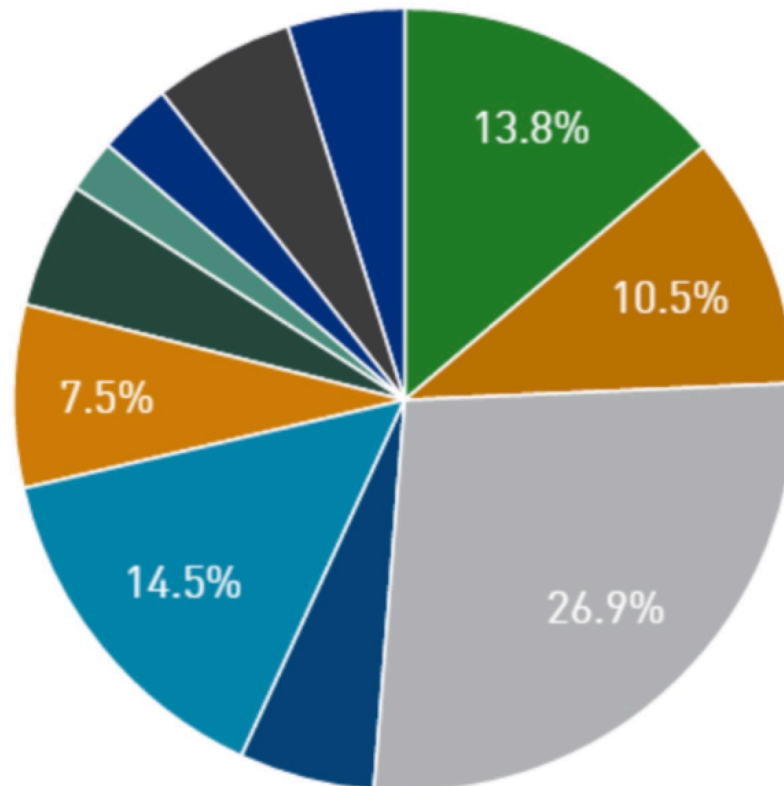
TOP500 Trend: Vendor

- CRAY和IBM仍占较大比例

Vendors System Share



Vendors Performance Share

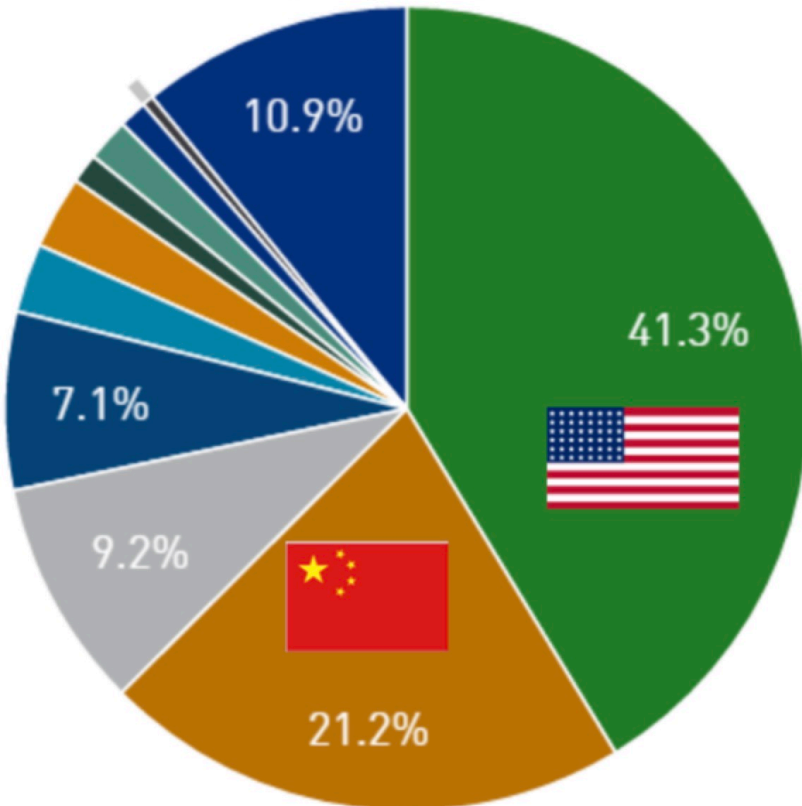




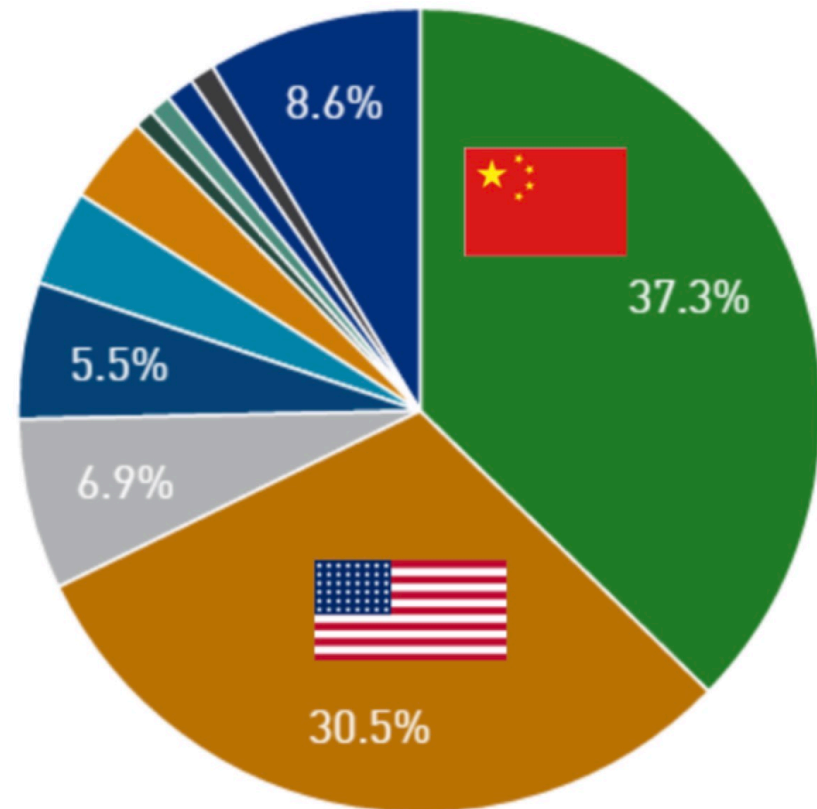
TOP500 Trend: Country

- 超计算机器的发明使我国跃至世界第二

2015 Nov



2016 June



Communication

- 对于超算，通信对并行程序的性能有着绝对的影响（比计算和访存更重要）
 - 网络通常比CPU更慢
 - 并行程序通信很常见
 - 同步操作非常昂贵





数据库中的并行计算：DOP

版本: latest

企业版 ☒ 轻量版

并行查询

UWAL

快速入门

安装指南

应用开发指南

编译指南

数据库管理指南

数据库运维指南

性能调优指南

文档地图 > 关于openGauss > 高级特性 > 并行查询

适用场景与限制

SMP特性通过算子并行来提升性能，同时会占用更多的系统资源，包括CPU、内存、I/O等等。本质上SMP是一种以资源换取时间的方式，在合适的场景以及资源充足的情况下，能够起到较好的性能提升效果；但是如果在不合适的场景下，或者资源不足的情况下，反而可能引起性能的劣化。SMP特性适用于分析类查询场景，这类场景的特点是单个查询时间较长，业务并发度低。通过SMP并行技术能够降低查询时延，提高系统吞吐性能。然而在事务类大并发业务场景下，由于单个查询本身的时延很短，使用多线程并行技术反而会增加查询时延，降低系统吞吐性能。

• 适用场景

- 支持并行的算子：计划中存在以下算子支持并行。
 - Scan：支持行存普通表和行存分区表顺序扫描、列存普通表和列存分区表顺序扫描。
 - Join：HashJoin、NestLoop
 - Agg：HashAgg、SortAgg、PlainAgg、WindowAgg（只支持partition by，不支持order by）。
 - Stream：Local Redistribute、Local Broadcast
 - 其他：Result、Subqueryscan、Unique、Material、Setop、Append、VectoRow
- SMP特有算子：为了实现并行，新增了并行线程间的数据交换Stream算子供SMP特性使用。这些新增的算子可以看做Stream算子的子类。
 - Local Gather：实现实例内部并行线程的数据汇总。
 - Local Redistribute：在实例内部各线程之间，按照分布键进行数据重分布。
 - Local Broadcast：将数据广播到实例内部的每个线程。
 - Local RoundRobin：在实例内部各线程之间实现数据轮询分发。
- 示例说明，以TPCH Q1的并行计划为例。

```
id | operation
---+-----
1 | -> Sort
2 | -> HashAggregate
3 | -> Streaming(type: LOCAL GATHER dop: 1/4)
4 | -> HashAggregate
5 | -> Seq Scan on lineitem
(5 rows)
```



数据库中的并行计算：DOP

产品

解决方案

客户

合作伙伴

资源与服务

文档

社区

请输入关键词

云控

OceanBase 数据库

SQL 型 - V4.2.1

分布式执行计划

分布式执行和并行查询

分布式计划的生成

分布式执行计划调度

分布式执行计划管理

并行开启方式及优先级

Auto DOP

并行查询的执行

并行查询的参数调优

并行 DML

并行 DDL

并行执行计划

SQL 优化

相关术语

SQL 调优常见问题

性能测试

数据库代理

文档中心 > OceanBase 数据库 > SQL 型 > V4.2.1 > 参考指南 > 性能调优 > SQL 调优指南 > 分布式执行计划 > Auto DOP

Auto DOP

更新时间：2024-11-19 17:15:41

什么是 Auto DOP

OceanBase 数据库优化器的并行度 DOP（Degree of Parallel）选择策略目前分为 `AUTO` 模式和 `MANUAL` 模式。选择 `AUTO` 模式即为 Auto DOP。

为什么选择 AUTO DOP?

您可以通过 `MANUAL` 模式手动指定并行度，一种方式是使用系统变量对当前会话中的所有执行查询开启并行执行并指定并行度，但是这样可能会给会话中本不需要并行加速，或者不需要使用较高 DOP 的查询带来额外的并行执行开销，从而导致性能下降；另一种方式是通过 Hint 来指定特定查询的并行度，但这需要对每一条业务查询进行单独考量，对于存在大量业务查询的场景是不可行的。

基于手动指定并行度所带来的不便和限制，`AUTO` 模式下的优化器可以根据查询实际情况，自动选择开启并行执行，并确定并行度。

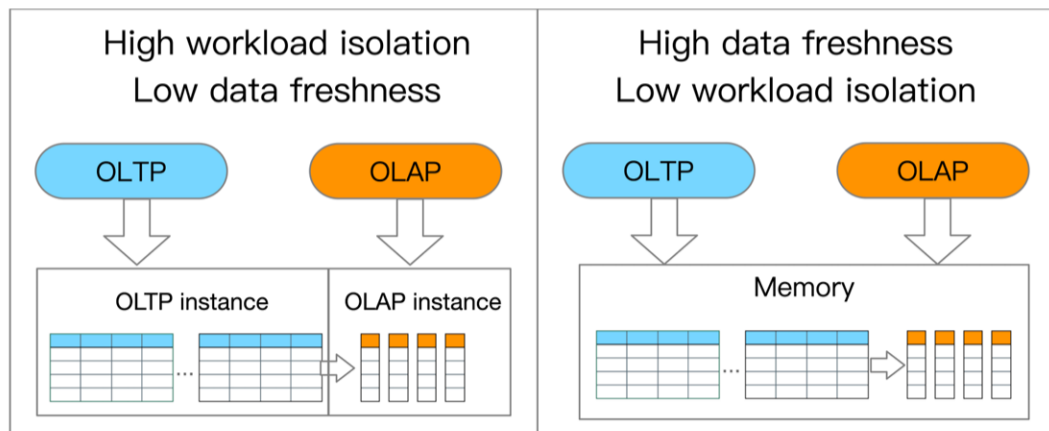
OceanBase 数据库查询优化器的 Auto DOP 功能，在生成查询计划时评估查询所需要执行的时间，自动确定是否开启并行以及确定对当前查询合适的并行度，避免了由于手动指定并行度而导致的性能下降问题。

如何开启 Auto DOP 策略?

OceanBase 数据库优化器可以通过系统变量 `parallel_degree_policy` 可以进行 `GLOBAL` 和 `SESSION` 级别的 Auto DOP 选择策略配置，也通过更高优先级的 Hint 进行查询级别的 DOP 选择策略配置。配置方法如下：

数据库中的并行计算：HTAP

- **Workload isolation**: the isolation level of handling the mixed workloads
- **Data freshness**: the portion of latest transaction data that is read by OLAP
- Trade-off for **workload isolation** and **data freshness**
 - High workload isolation leads to low data freshness
 - Low workload isolation results in high data freshness



数据库中的并行计算：GPU

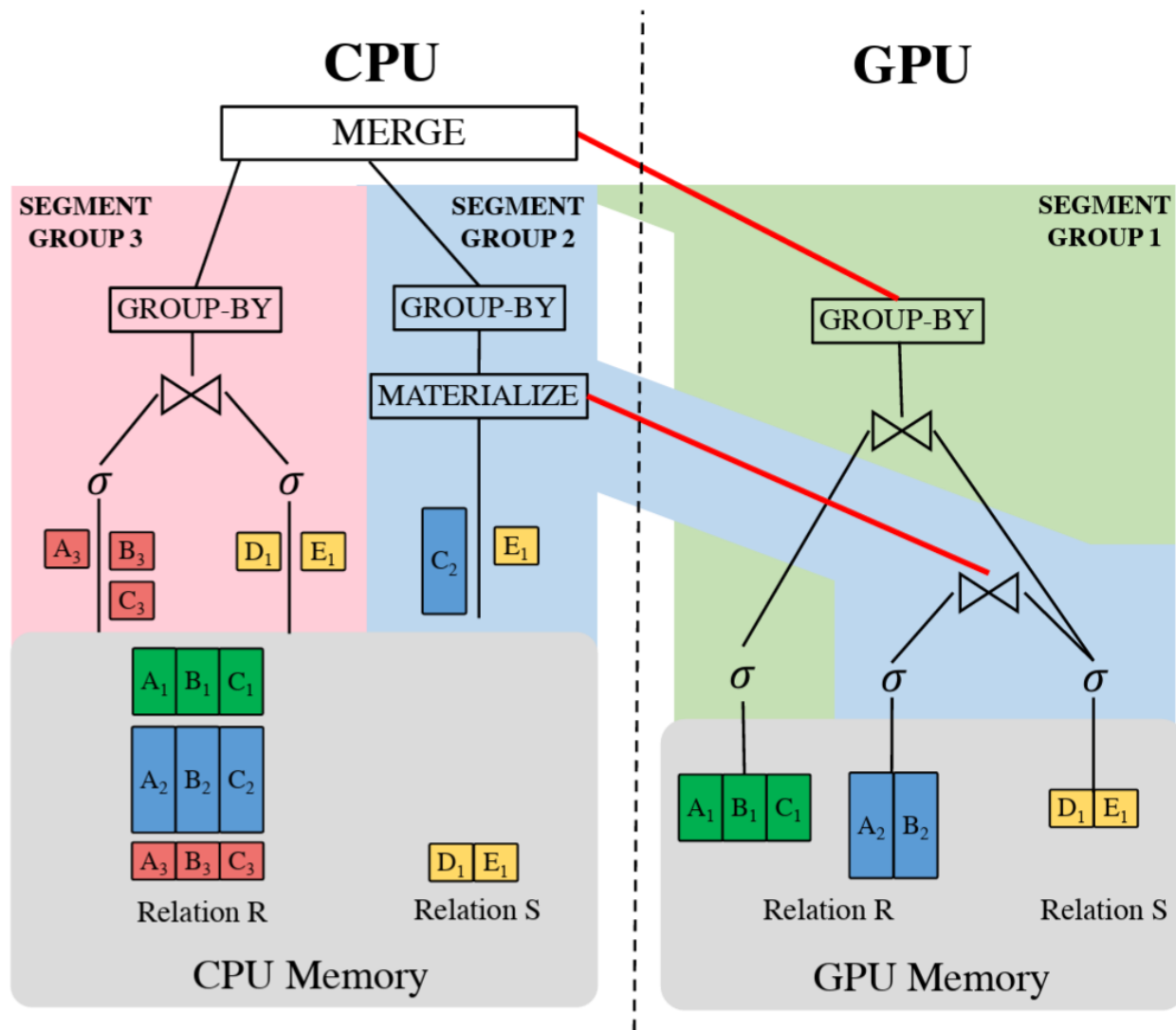


Figure 3: Example of Segment-level Query Plan.



目录

- 并行计算介绍
- 并行计算机和编程模型的分类
 - 弗林（**Flynn**）经典分类
 - 存储系统分类
 - 编程模型分类
- 并行计算基本概念与数据模型
- 并行程序分析



并行计算机分类

- 弗林（**Flynn**）的经典分类
 - 1966年（50年前）
 - 从处理单元的角度：从**指令**和**数据**角度对计算机体系结构分类

SISD

Single Instruction

Single Data

SIMD

Single Instruction

Multiple Data

MISD

Multiple Instruction

Single Data

MIMD

Multiple Instruction

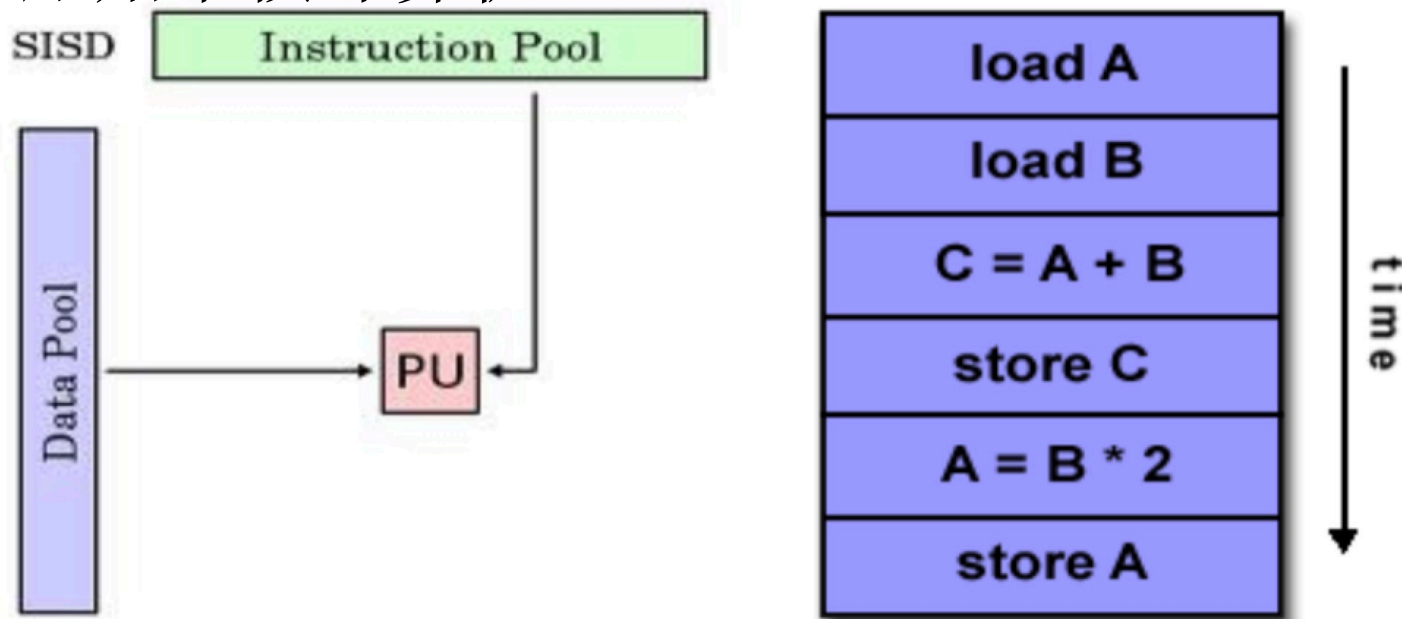
Multiple Data



弗林（Flynn）的经典分类

SISD

- Single Instruction, Single Data (SISD):
 - 顺序执行计算机（非并行）
 - 单指令：CPU每个时钟周期处理一个指令流
 - 单数据：每个时钟周期只处理一个数据流
 - 原始单核计算机

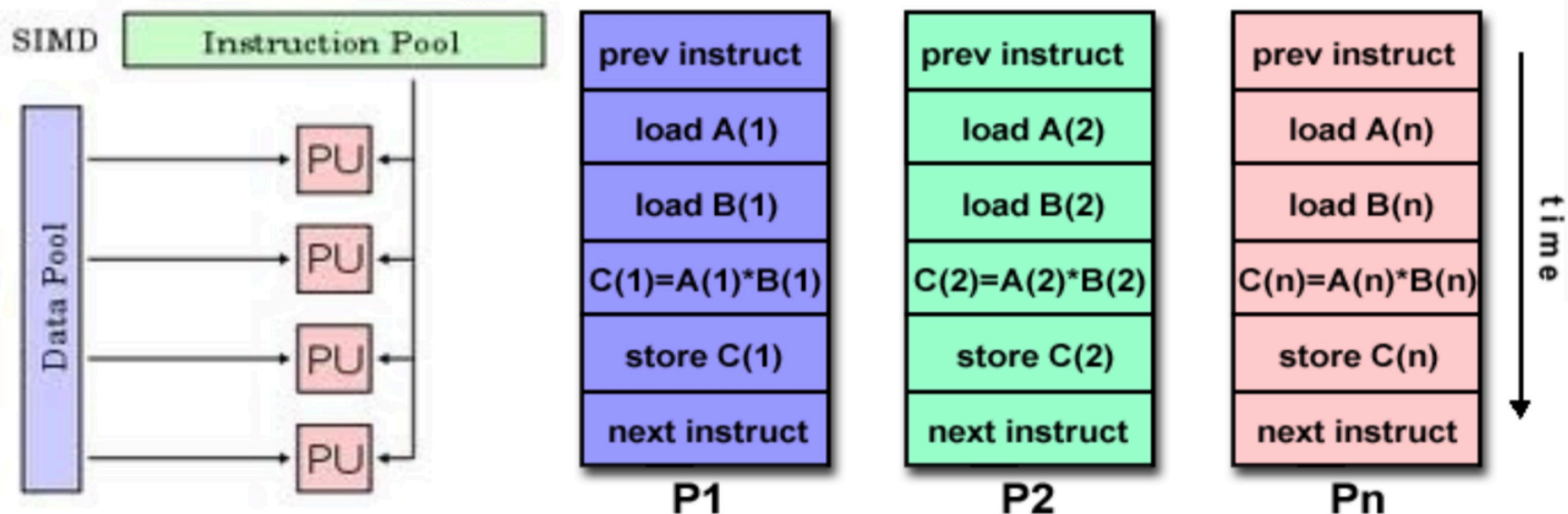




Flynn的经典分类

SIMD

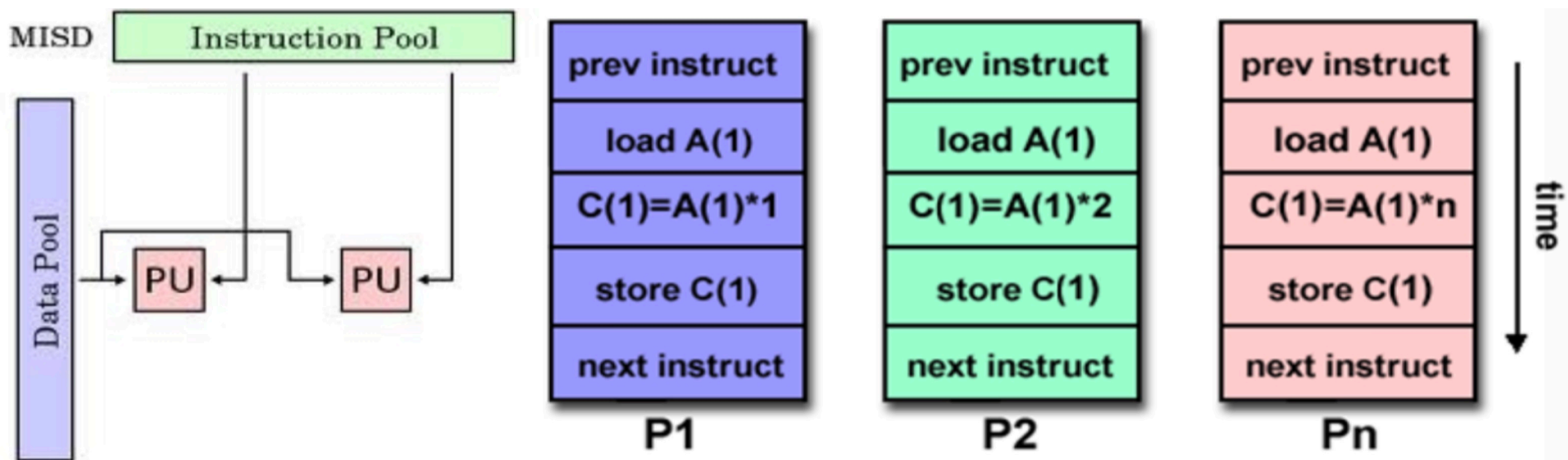
- Single Instruction, Multiple Data (SIMD):
 - 单指令：所有的处理单元同一时钟周期都执行同样的指令
 - 多数据：每个处理单元可以操作不同的数据单元
 - 例子：GPU，向量处理器（X86 AVX instruction）



Flynn的经典分类

MISD

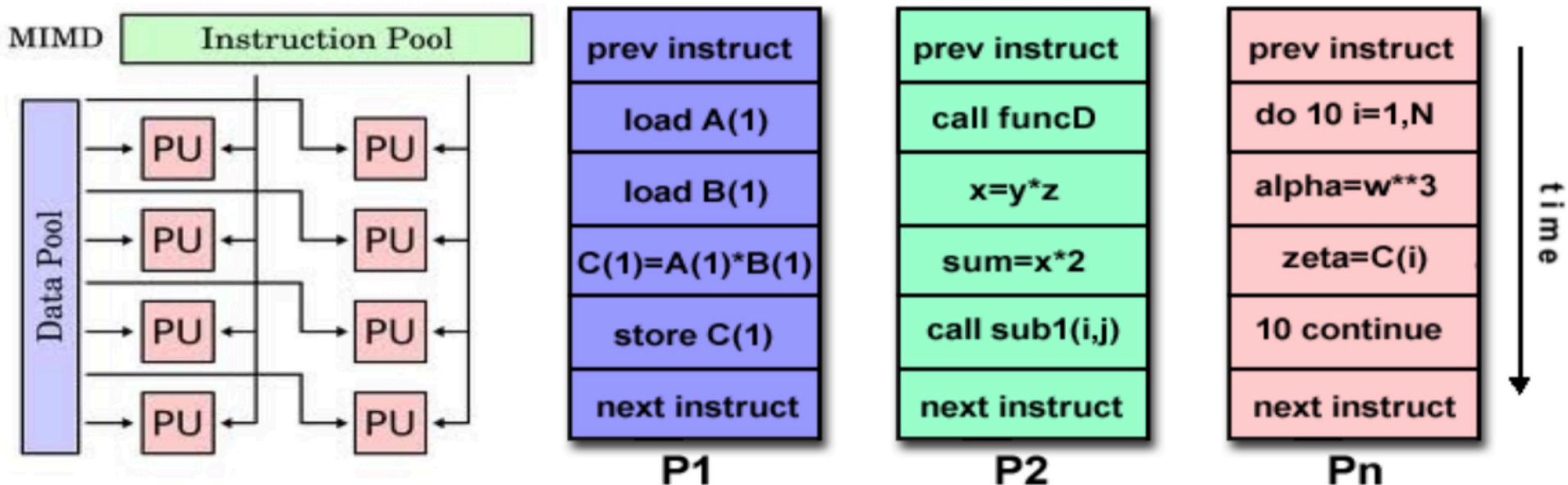
- Multiple Instruction, Single Data (MISD):
 - 多指令：每个处理单元可以彼此独立地通过指令流对数据进行处理
 - 单数据：单数据流可以由多个处理单元获取
 - 例子：CMU在1971年提出的概念机器，可用于fault tolerance



Flynn的经典分类

MIMD

- Multiple Instruction, Multiple Data (MIMD):
 - 多指令：每个处理器可以执行不同的指令流
 - 多数据：每个处理器可以处理不同的数据流
 - 例子：现代多核处理器





目录

- 并行计算介绍
- 并行计算机和编程模型的分类
 - Flynn的经典分类
 - 存储系统分类
 - 编程模型分类
- 并行计算基本概念与数据模型
- 并行程序分析

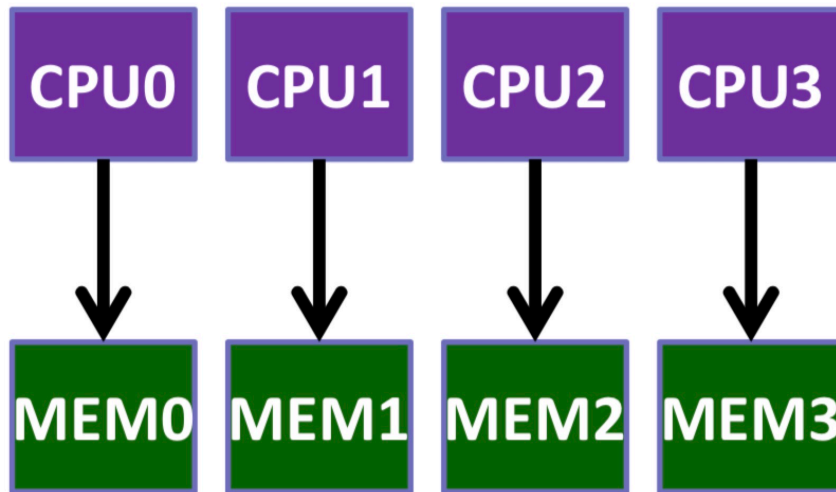


思考

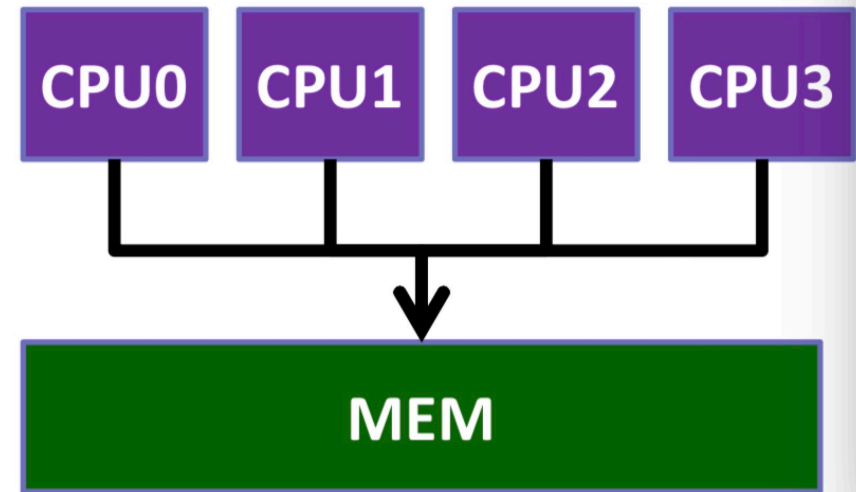


- Shared Memory 和 Distributed Memory 特点是什么？
- NUMA和UMA有什么区别？

Shared Memory vs. Distributed Memory Computer Architecture



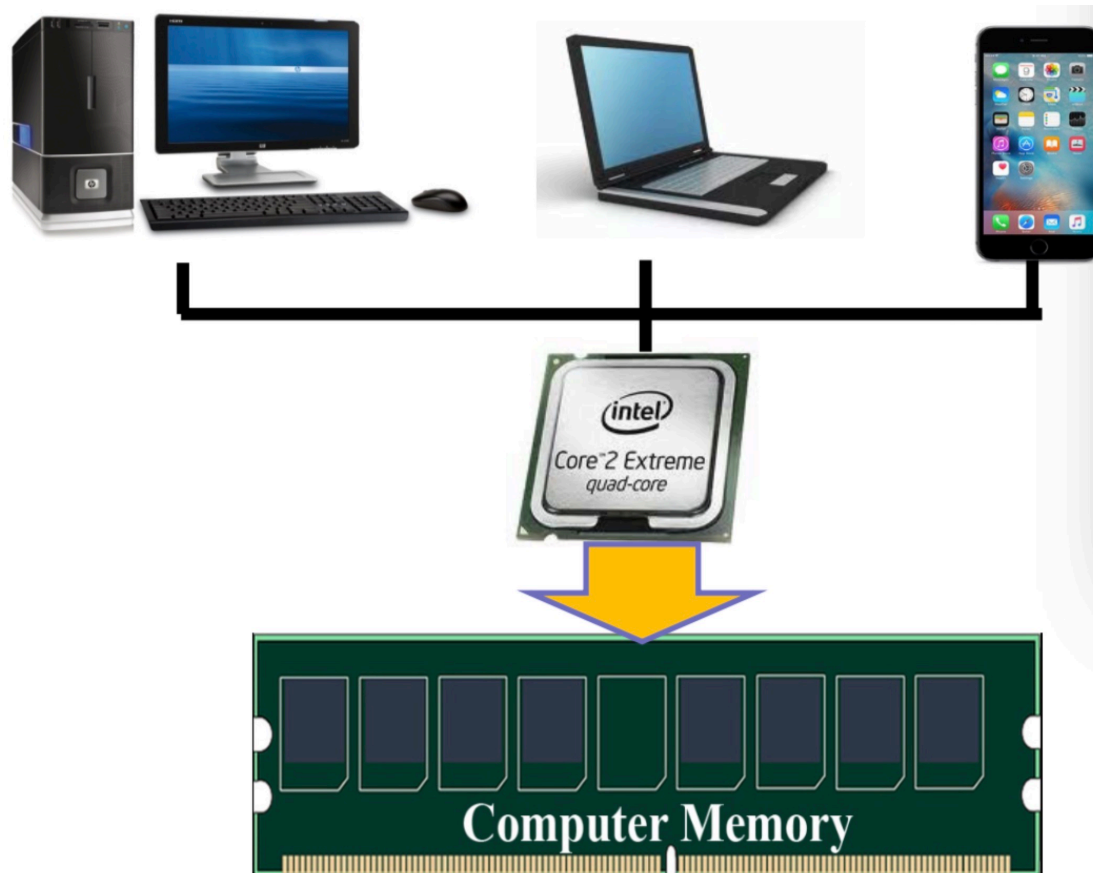
Distributed memory



Shared memory

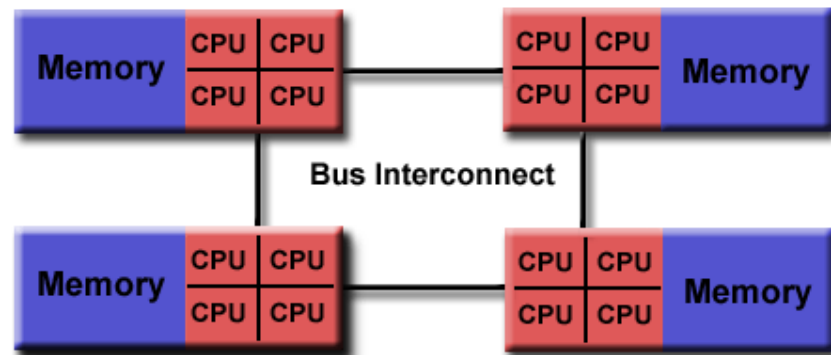
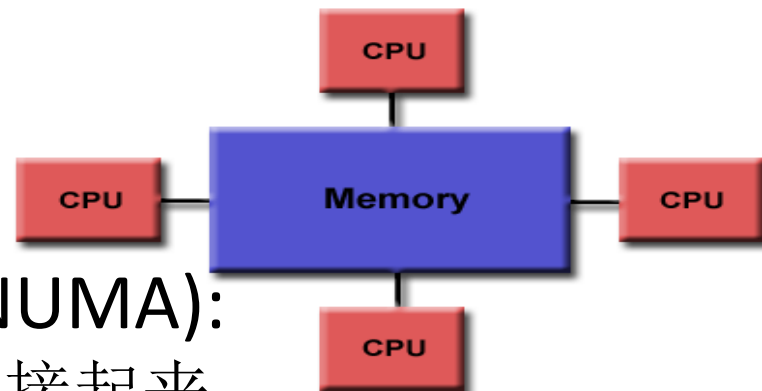
Shared Memory Multiprocessor Computer System

- Single computer with multiple internal multi-core processors



Shared Memory Computer Architecture

- Uniform Memory Access (UMA):
 - 最常见于Symmetric Multiprocessor (SMP)机器
 - 独立处理器
 - 访问内存时间相同
 - 例子：商用服务器
- Non-Uniform Memory Access (NUMA):
 - 通常将多个SMP通过物理链路连接起来
 - 一个SMP可以直接访问另一个SMP的内存
 - 跨链路内存访问慢
 - 例子：HPC服务器

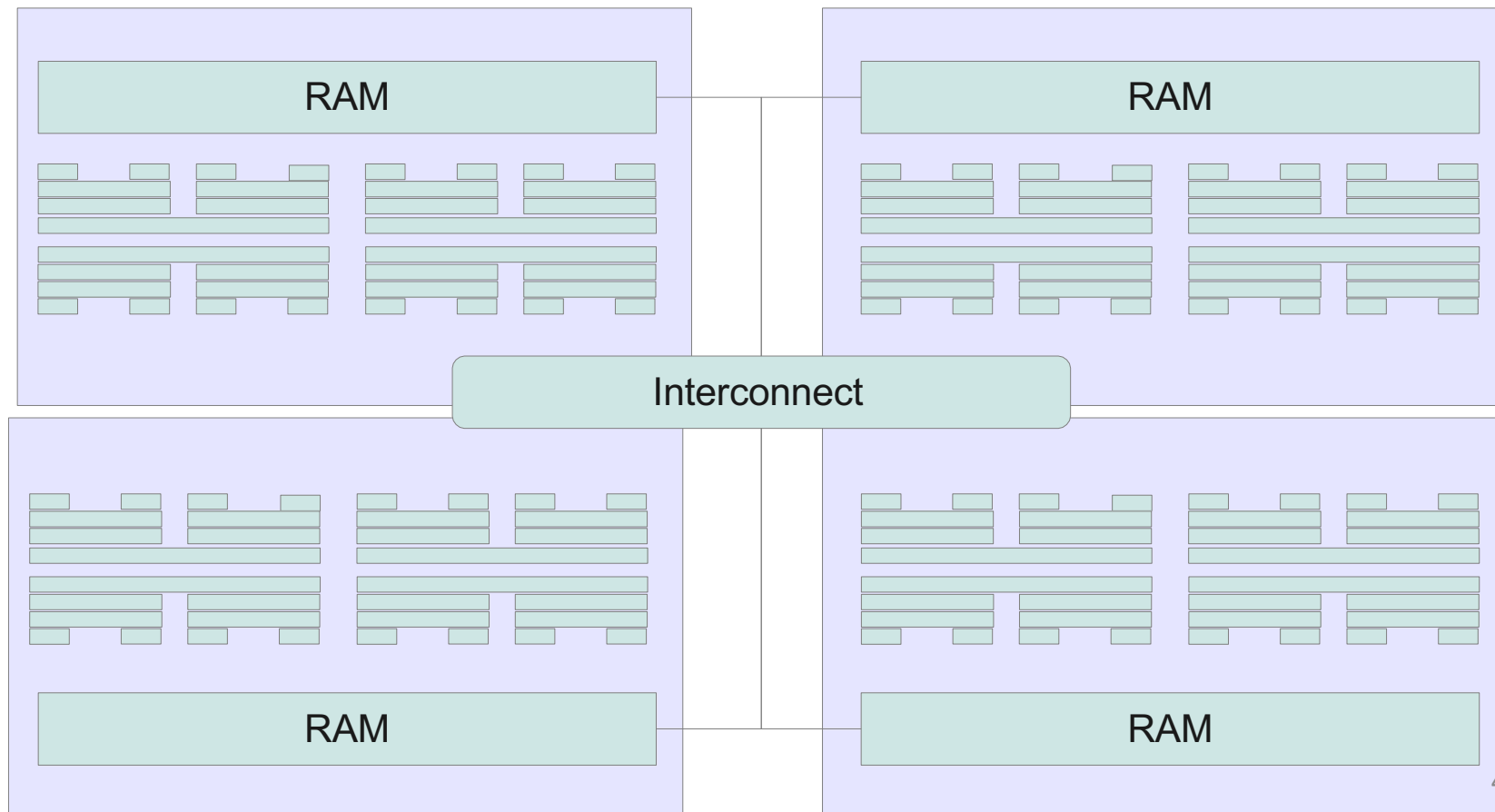




并行体系结构

- NUMA (Non-Uniform Memory Access) architecture.
- Linking several SMPs (Symmetric Multi-Processor).
- Coherency not maintained.

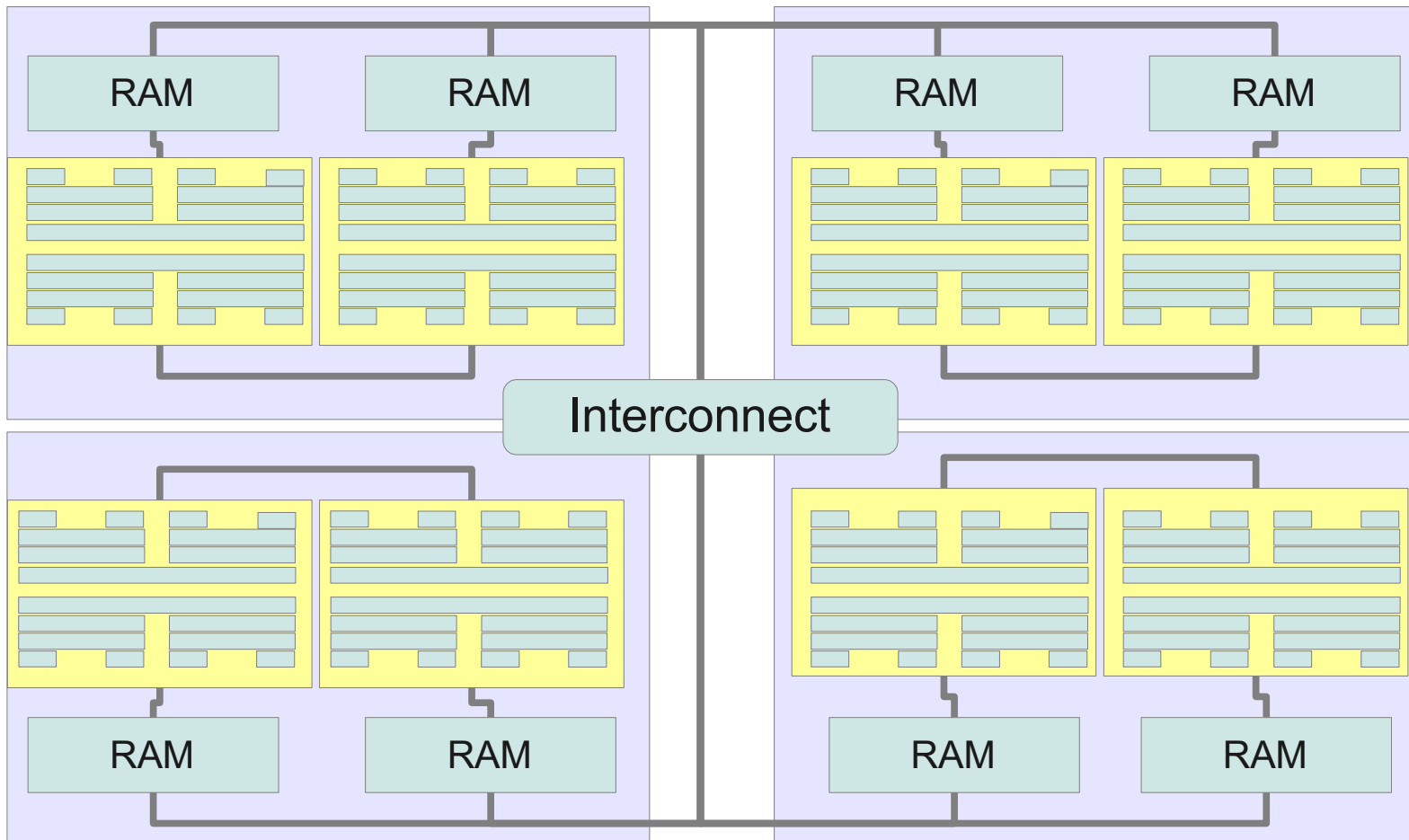
Distributed
Memory



并行体系结构

- ccNUMA (cache coherent Non-Uniform Memory Access) architecture.
- Communication between cache controllers to maintain coherency.
- Consistent memory image.

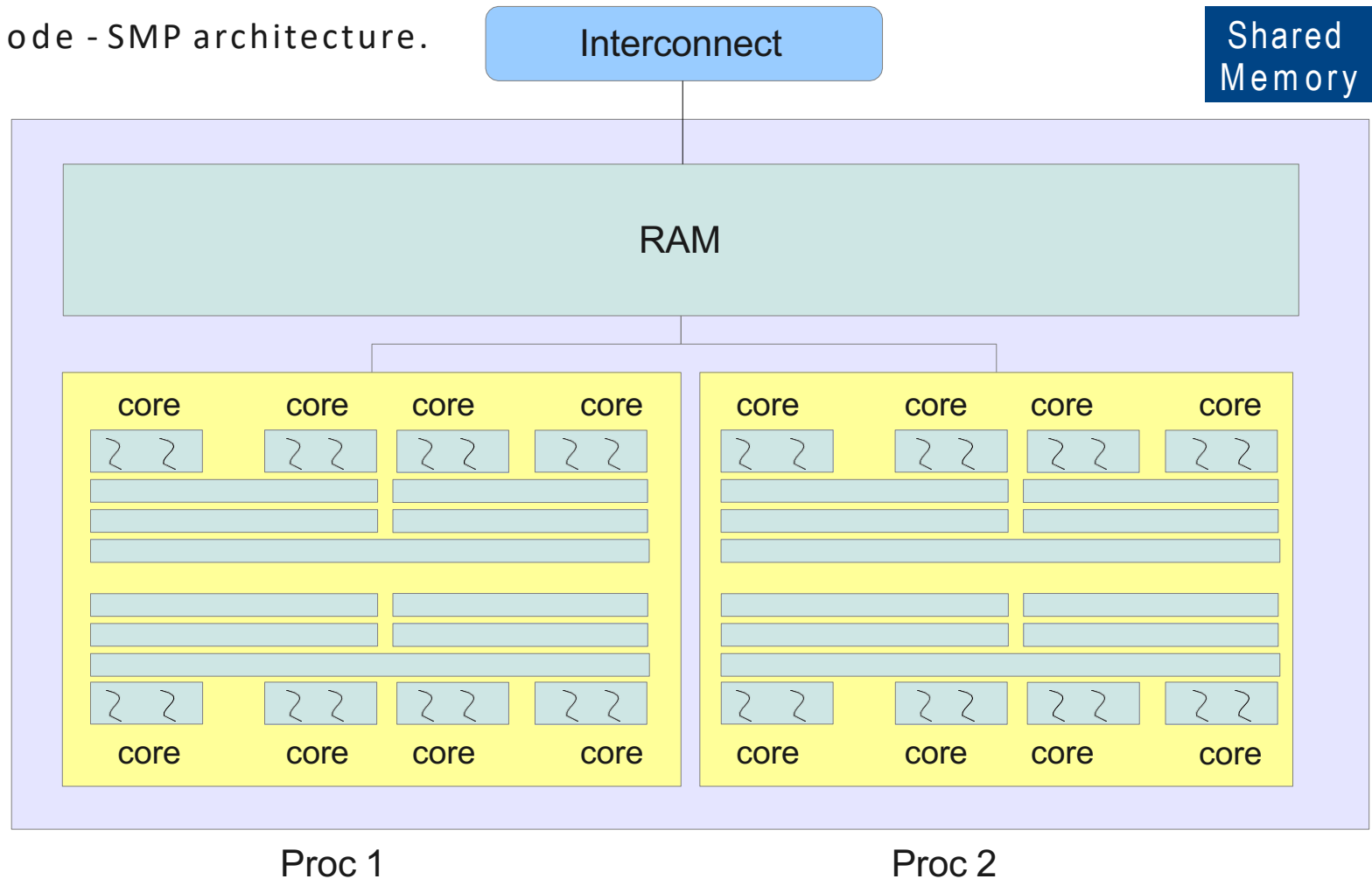
Distributed
Memory





并行体系结构

NUMA node - SMP architecture.



Distributed Memory Multicomputer

- 通过将多个计算机连接起来构成一个计算平台，无需共享内存



Cluster: tens of servers



Supercomputer:
hundreds of servers



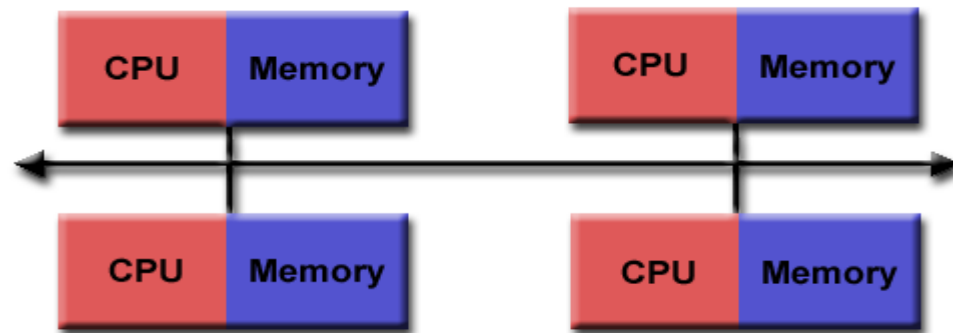
Datacenter: thousands of servers

Distributed Memory Multicomputer



- 需要通信网络连接处理器内存
- 各处理器拥有独立的内存和地址空间
- 各处理器间内存改变不会互相影响
- 程序员或编程工具负责显式定义数据是如何在处理器间传递的

Network fabric:
Ethernet,
InfiniBand





目录

- 并行计算介绍
- 并行计算机和编程模型的分类
 - Flynn的经典分类
 - 存储系统分类
 - 编程模型分类
- 并行计算基本概念与数据模型
- 并行程序分析



思考

- 共享内存模型和消息传递模型的区别是什么？
- 共享内存模型的两种常见的实现方法是什么？

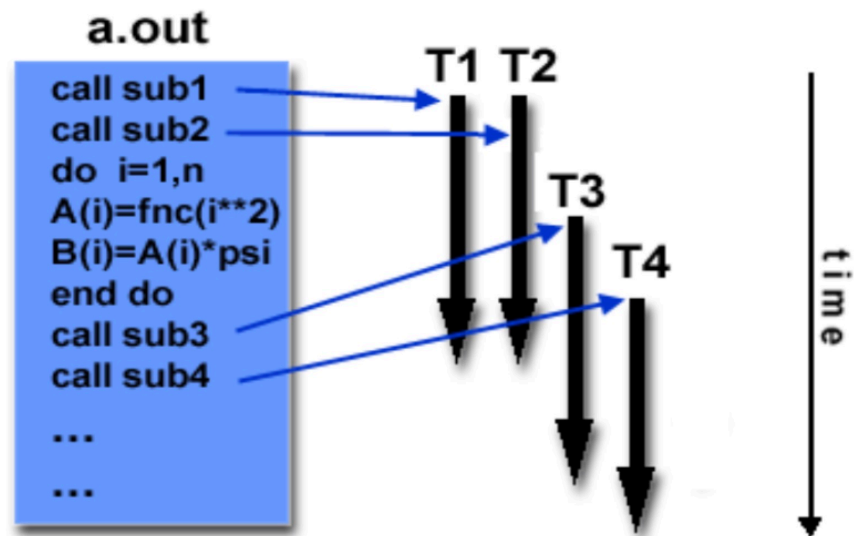
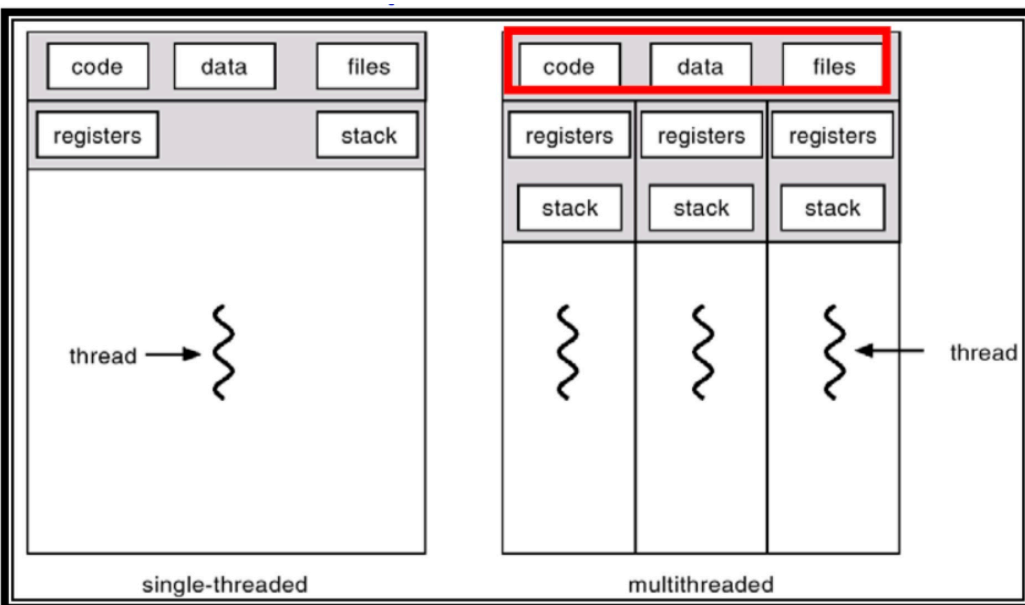


Parallel Programming Model

- 并行编程模型是硬件和存储体系结构之上的一层抽象
- 通常编程模型的设计会匹配计算机体系结构
 - 共享内存编程模型 – 共享内存机器
 - 消息传递编程模型 – 分布式内存机器
- 但编程模型并不会严格和机器、存储体系结构对应
 - 共享内存编程模型也可以支持分布式内存机器，MPI
 - 消息传递编程模型也可以支持共享内存机器

Shared Memory Programming Model

- 单个进程可以有多个并行执行路径
- 线程可以有局部数据，同时有共享资源
- 线程间通过全局内存通信
- 线程可以被创建、销毁，但主线程一直在





Shared Memory Programming Model

实现

- 并行程序代码调用**库函数**
 - 例子: POSIX Threads (Pthreads)
- **编译指导语句**嵌入在串行/并行代码中
 - 例子: OpenMP

```
#include <pthread.h>
void print_message_function ( void *ptr ) { }
printf("Hello, world.\n");
int main() {
    pthread_t thread;
    pthread_create (&thread, NULL, (void *)
    &print_message_function, NULL);
    pthread_join(thread, NULL);
}
```

```
#include <omp.h>
int main() {
    #pragma omp parallel
    {
        printf("Hello, world.\n");
    }
}
```

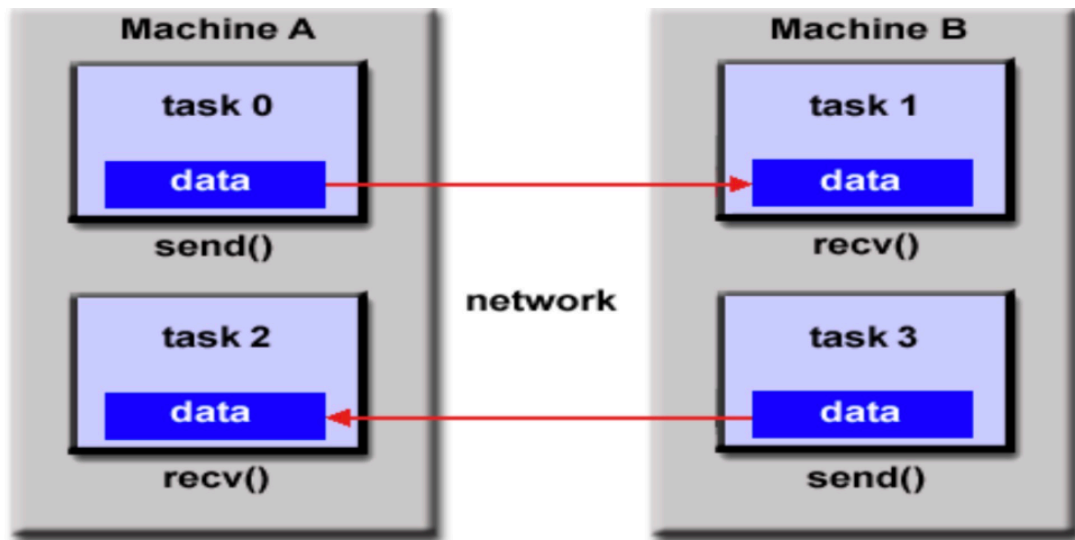


Message Passing Programming Model

- 一组任务，在计算时使用各自**局部存储**
 - 多个任务可以在同一个物理机器上，也可以在不同的机器上
- 任务间通过**发送和接收消息**交换数据
(memory copy)

■ MPI API:

- Send, Recv, Bcast, Gather, Scatter, etc.





共享内存 VS 消息传递

- 共享内存
- 方便：
 - 可以共享数据结构
 - 通过loop实现
 - 和串行代码接近
- 缺点：
 - 局部性不可控
 - 可扩展性差
 - 访问冲突
- 消息传递
- 扩展性好：
 - 局部性控制
 - 手工显式实现传输
- 缺点：
 - 需要设计应用/数据结构
 - 复杂的代码



总结

- 编程模型和并行系统的设计和流程度彼此互相影响
- OpenMP, MPI, Pthreads, CUDA是一些常见的并行语言，用于进行并行编程
- 了解并行计算思想比了解如何进行并行编程更重要，因为这样可以：
 - 很快掌握一个新的并行编程工具
 - 了解程序的性能
 - 优化程序的性能



目录

- 并行计算介绍
- 并行计算机和编程模型的分类
- 并行计算基本概念
 - 加速比基本定律与性能极限分析
 - 并行数据结构设计
 - 并行算法设计与复杂度分析



思考



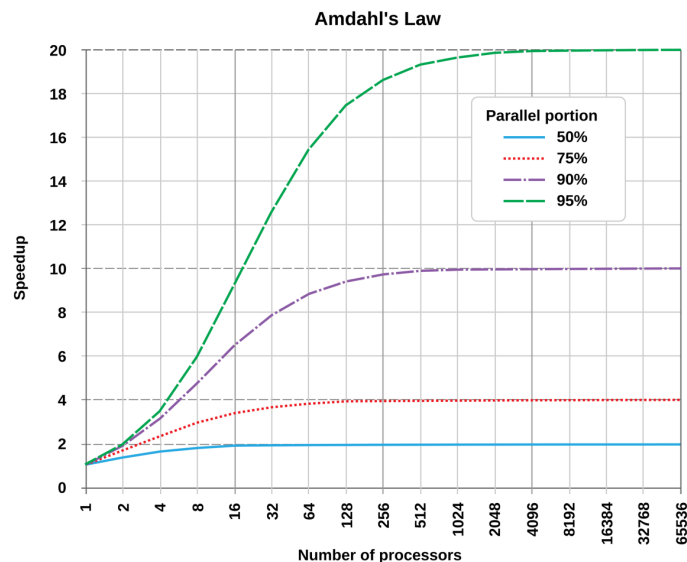
- 如何能够知道并行计算的极限加速比？
- 数据结构如何影响并行算法？

基本定律

- 阿姆达尔（Amdahl）定律: 并行计算的极限受到程序串行部分的限制

- $\text{Speedup}(N) = 1 / (S + P/N)$
- 其中S是串行工作比例，P是并行工作比例，并且 $S + P = 1$ ，N是处理器数目

核心观点：问题规模不变

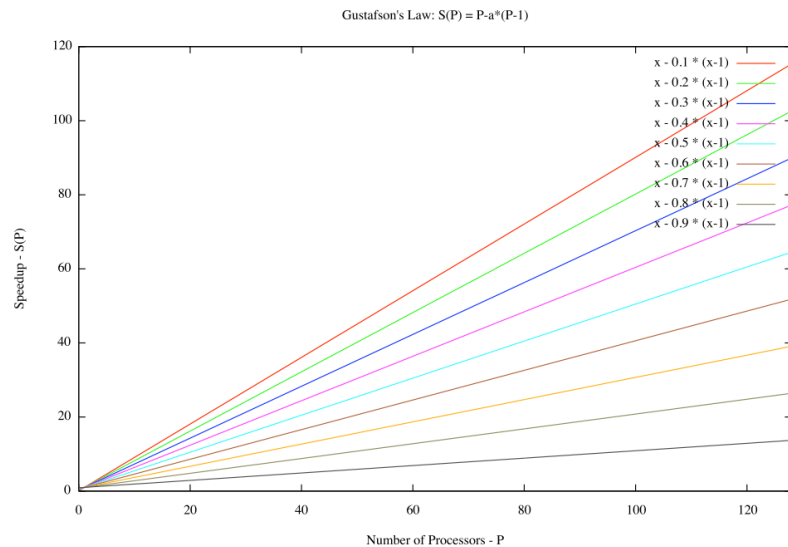


基本定律

- 古斯塔夫森（Gustafson-Barsis）定律:对代码使用并行技术可以处理更大规模的问题

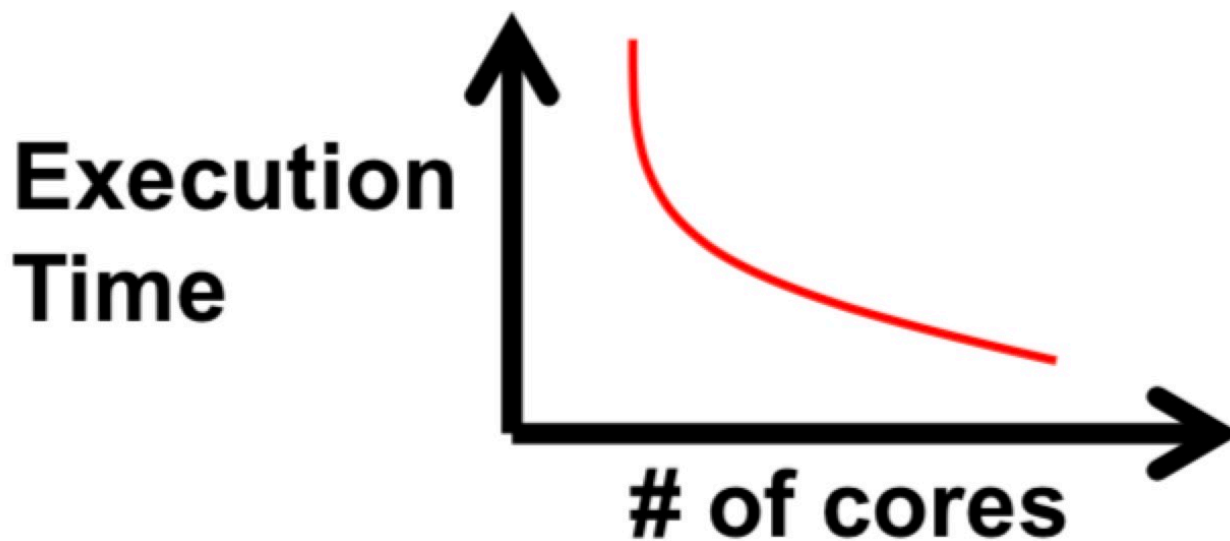
- $\text{Speedup}(N) = N - S(N-1)$
- 其中S是串行工作比例
- N是处理器数目

核心观点：问题规模变化



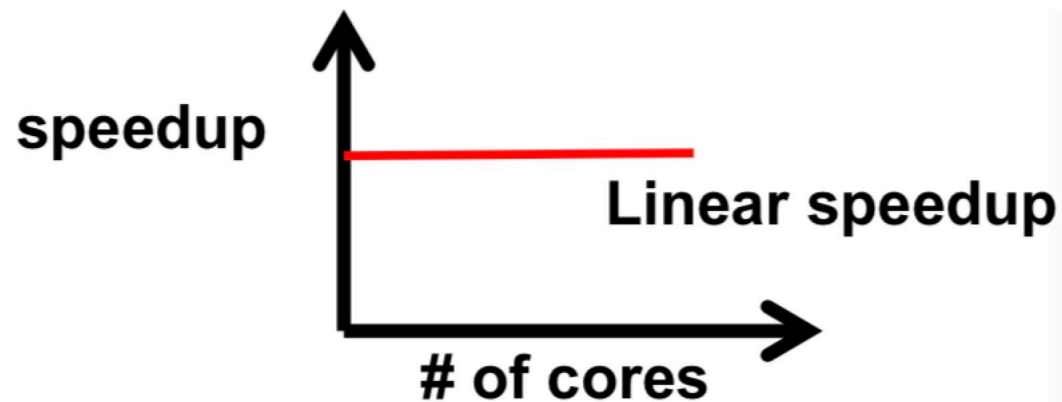
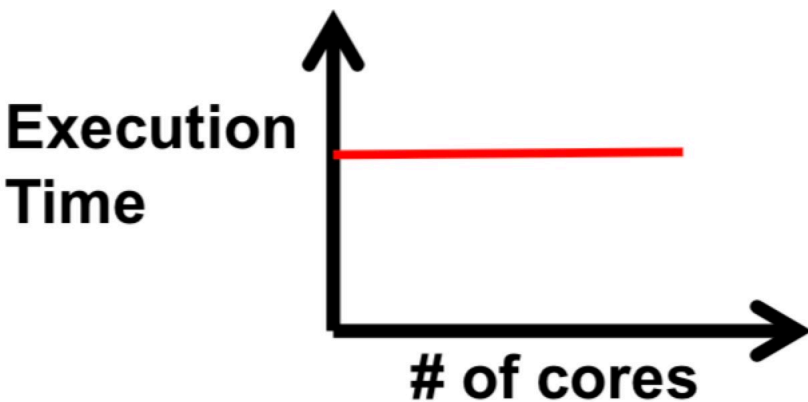
Strong Scaling

- 问题规模固定，处理单元数目增加
- 用于找到一个特定的点，在合理的时间范围内完成计算任务，但又不至于浪费太多计算资源



Weak Scaling

- 每个处理单元处理的规模是固定的，增加计算单元来解决更大规模的问题
- 当负载增加各处理单元运行时间保持不变时可获得线性扩展比





硬件性能极限理论值分析

- 了解程序的潜在性能限制
- 执行浮点运算的速率 (**flop/s**)
- 数据在不同级别内存之间移动的速率 (**GB/s**)
- 硬件运行应用程序时所消耗的功率 (**W**)
- 网络传输的延迟 (**s**)

NB: 计算机科学家往往认为浮点运算 (**flop**) 是主要的性能限制, 但现实真正的限制来自于**带宽和延迟**



计算浮点运算的最大理论值

- MacBook Pro配置如下：
- Intel Core i7-7920HQ，四核处理器，可达3.7GHz，单个处理器支持2个超线程，向量处理频率为8flop/周期，请问最大的浮点计算理论值（F）如何计算？
- $F = \text{虚拟核心数} * \text{时钟频率} * \text{向量处理频率}$



内存带宽的最大理论值

- MacBook Pro配置如下：
- LPDDR3-2133内存（2133 MT/s），双通道，每次访问8个字节数，请问最大内存带宽理论值（B）如何计算？
- $B = \text{数据传输速率} * \text{内存通道数} * \text{每次访问字节数} * \text{插槽数量}$

数据结构与并行计算

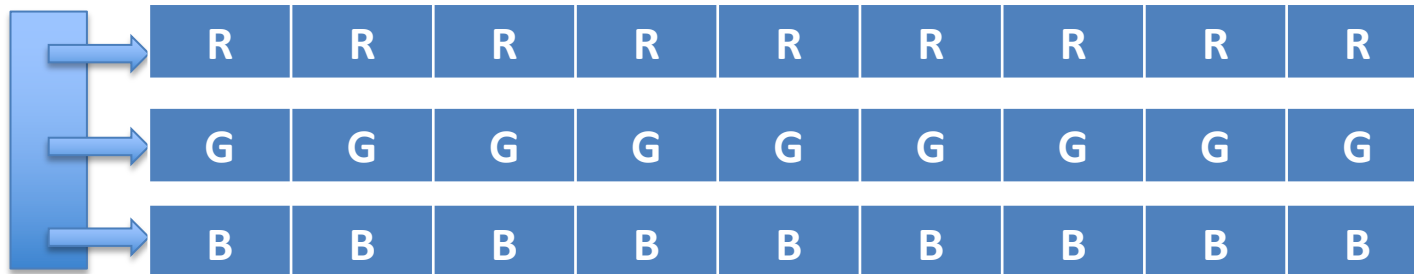
- 结构数组 (AoS)

- Struct RGB{int R; int G; int B};
- Struct RGB polygon_color[1000];



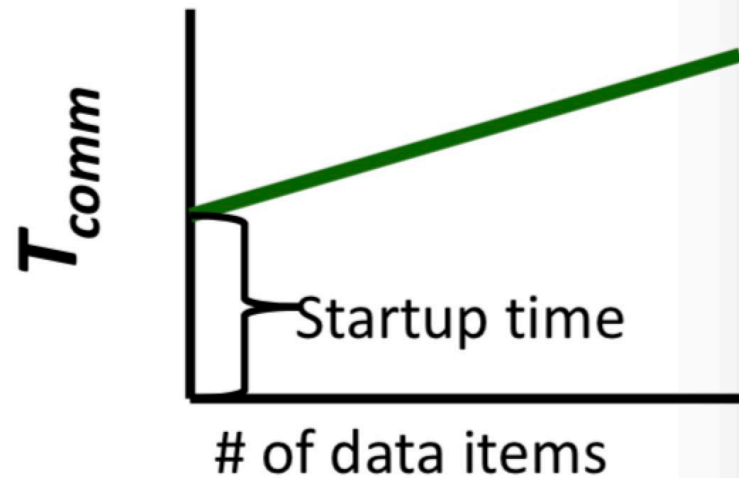
- 数组结构 (SoA)

- Struct RGB{int *R; int *G; int *B};
- Struct RGB polygon_color;
- polygon_color.R=(int *) malloc(1000*sizeof(int));



Time Complexity Analysis

- $T_p = T_{comp} + T_{comm}$
- T_p : 并行算法总执行时间
- T_{comp} : 计算部分
- T_{comm} : 通信部分
- $T_{comm} = q(T_{startup} + n * T_{data})$
 - $T_{startup}$: 消息延迟（假设为固定值）
 - T_{data} : 发送一个数据项的传输时间
 - n : 消息中的数据项数目
 - q : 消息的数目





Time Complexity Example 1

- 算法：
 - 1. 节点1 发送 $n/2$ 个数字至节点2
 - 2. 两个节点同时将各自的 $n/2$ 个数字相加
 - 3. 节点2将相加后的结果发送回节点1
 - 4. 节点1将收到的结果和自己的结果相加得到最终结果
- 复杂度分析：



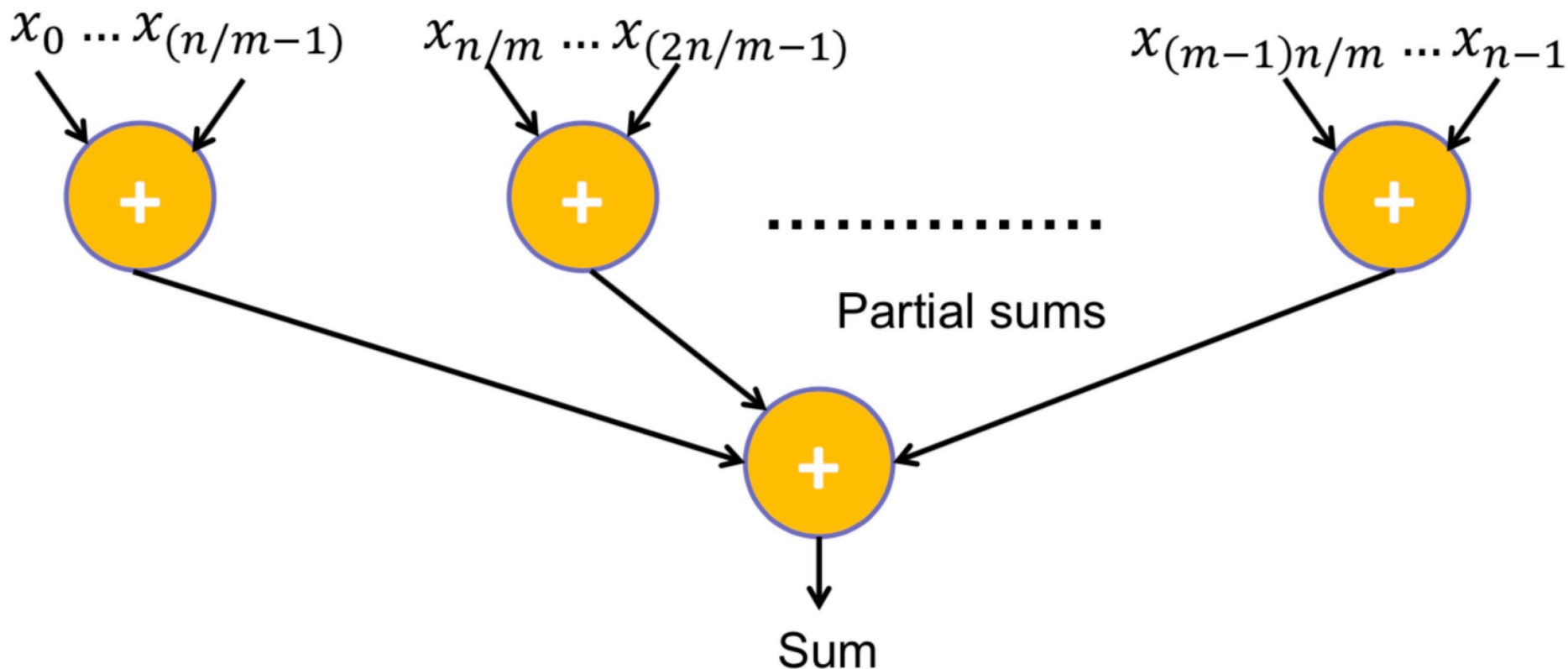
Time Complexity Example 1

- 算法：
 - 1. 节点1 发送 $n/2$ 个数字至节点2
 - 2. 两个节点同时将各自的 $n/2$ 个数字相加
 - 3. 节点2将相加后的结果发送回节点1
 - 4. 节点1将收到的结果和自己的结果相加得到最终结果
- 复杂度分析：
 - 计算部分（步骤2&4）：
 - $T_{\text{comp}} = n/2 + 1 = O(n)$
 - 通信部分（步骤1&3）：
 - $T_{\text{comm}} = (T_{\text{startup}} + n/2 * T_{\text{data}}) + (T_{\text{startup}} + T_{\text{data}})$
 - $= 2T_{\text{startup}} + (n/2 + 1) * T_{\text{data}} = O(n)$

复杂度： $O(n)$

Time Complexity Example 2

- 将 n 个数字用 m 个进程做加和
 - 均匀地分配个每个进程去做





Time Complexity Example

- 串行: $O(n)$
- 并行:
 - 1. 发送数字到对应的进程
 - $T_{comm1} = m(T_{startup} + (n/m)T_{data})$
 - 2. 计算局部和
 - $T_{comp1} = n/m - 1$
 - 3. 发送结果回主进程
 - $T_{comm2} = m(T_{startup} + T_{data})$
 - 4. 计算最终结果
 - $T_{comp2} = m - 1$
 - 最终:
 - $T_p = 2mT_{startup} + (n+m)T_{data} + m + n/m - 2 = O(m + n/m)$

计算和通信
之间做权衡



Cost-Optimal Algorithm

- 定义：
 - 求解一个问题的时间和使用单处理器求解问题的时间成比例
 - $O(T_p) * N = O(T_s)$
- 例子：
 - 算法串行时间： $O(N \log N)$
 - 并行算法1： 使用N个处理器 $O(\log N)$
 - 并行算法2： 使用 $N * N$ 个处理器 $O(1)$



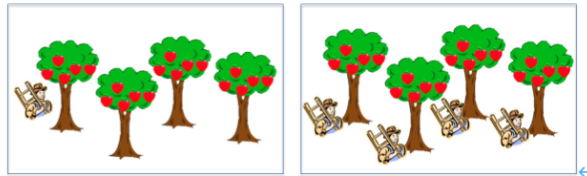
第一次课总结

- 并行计算介绍：弗林经典分类
- 并行计算模型：存储模型与编程模型
- 并行计算定律：阿姆达尔与古斯塔夫森定律
- 并行程序分析：理论最大值与复杂度分析

课程作业一

思考 1: 生活中有什么并行处理的例子? ↵

围绕下图讲一讲并行是如何提高效率的? ↵



答: ↵

↵

↵

思考 2: Shared Memory 和 Distributed Memory 特点是什么? ↵

NUMA 和 UMA 有什么区别? ↵

答: ↵

↵

↵

思考 3: 共享内存模型和消息传递模型的区别是什么? ↵

共享内存模型的两种常见的实现方法是什么? ↵

答: ↵

↵

↵

思考 4: 为什么会有加速器的产生? ↵

答: ↵

↵

↵

思考 5: Linear array, Ring, Tree, Mesh, Torus, Hypercube 拓扑结构的优缺点? 适用于哪些场合? ↵

答: ↵

↵

↵

思考 6: 假设串行程序的执行时间是 10 秒, 通过使用 4 个计算核心使得程序执行时间降为了 5 秒, 请问: ↵

程序的加速比是多少? ↵

系统效率是多少? ↵

答: ↵



课程实验一

- 配置OpenMP环境: **VS Code**
- 利用多线程输出多个: **Hello World**

```
#include <omp.h>
int main() {
    #pragma omp parallel
    {
        printf("Hello, world.\n");
    }
}
```